

번역 원칙: 본문은 한국어로 옮기되, 고유명사·모델명·기술 용어·변수·수식·논문명은 원문의 English 표기를 유지했습니다.
도표·표의 수치와 시각 요소는 원본을 보존하고, 모든 캡션은 한국어로 번역했습니다.

mechanistic interpretability를 통한 Grokking의 Progress Measure

원문: Progress Measures for Grokking via Mechanistic Interpretability
ICLR 2023 conference paper

Neel Nanda^{* †} Lawrence Chan[‡] Tom Lieberum[†] Jess Smith[†] Jacob Steinhardt[‡]

* Corresponding author: neelnanda27@gmail.com † Independent researcher ‡ University of California, Berkeley

초록

Neural network는 parameter 수, training data, 또는 training step 수가 커질 때 질적으로 새로운 능력이 나타나는 emergent behavior를 자주 보인다. emergence를 이해하는 한 접근은, 겉보기에는 불연속적인 질적 변화를 뒷받침하는 연속적인 progress measure를 찾는 것이다. 본 논문은 mechanistic interpretability, 즉 학습된 행동을 개별 구성요소로 reverse-engineering하는 방법을 통해 progress measure를 찾을 수 있다고 주장한다.

사례 연구로, modular addition task에서 학습된 작은 transformer가 보이는 최근 발견된 현상인 “grokking”을 조사한다. 이 network가 학습한 algorithm을 완전히 reverse engineer하며, 이 algorithm은 discrete Fourier transform와 trigonometric identity를 사용하여 덧셈을 원 위의 회전으로 바꾼다. activation과 weight를 분석하고 Fourier space에서 ablation을 수행하여 이 algorithm을 확인한다. 이러한 이해를 바탕으로 training dynamics를 연구할 수 있는 progress measure를 정의하고, training을 memorization, circuit formation, cleanup이라는 세 연속 단계로 나눈다. 결과는 grokking이 갑작스러운 전환이라기보다, weight에 encode된 구조화된 mechanism이 점진적으로 증폭되고 이후 memorization component가 제거되면서 발생함을 보여 준다.

1. INTRODUCTION

Neural network는 model size, training data, 또는 training step 수가 증가할 때 질적으로 새로운 능력이 나타나는 emergent behavior를 자주 보인다(Steinhardt, 2022; Wei et al., 2022a). 이는 in-context learning(Radford et al., 2019; Brown et al., 2020), chain-of-thought prompting(Wei et al., 2022b) 같은 능력을 통해 여러 breakthrough를 낳았다. 그러나 위험도 있다. Pan et al. (2022)은 model의 parameter 수를 30% 정도만 늘려도 emergent reward hacking이 나타날 수 있음을 보였다.

reward hacking, chain-of-thought reasoning, 또는 다른 phase transition처럼 emergence가 갑작스러울 때 특히 놀랍다(Ganguli et al., 2022; Wei et al., 2022a). phase transition보다 먼저 나타나고 그와 인과적으로 연결되며 더 smooth하게 변하는 metric, 즉 hidden progress measure(Barak et al., 2022)를 찾는다면 이러한 전환을 더 잘 이해하고 예측할 수 있다. 예를 들어 Wei et al. (2022a)은 large language model이 여러 benchmark에서 성능의 급격한 도약을 보이지만 cross-entropy loss는 model scale에 따라 smooth하게 감소함을 보였다. 하지만 cross-entropy 자체는 phase change가 왜 일어나는지 설명하지 못한다.

본 연구는 hidden progress measure를 찾아내는 다른 접근, 즉 mechanistic explanation을 제시한다. mechanistic explanation은 일반적으로 특정 행동을 구현하는 model 내부의 circuit(Cammarata et al., 2020; Elhage et al., 2021)을 식별하여 network mechanism을 reverse engineer하려는 설명이다. 이를 이용하여, model이 처음에는 overfit하지만 많은 training step 뒤 generalizing solution으로 급격히 전환되는 grokking을 연구한다(Power et al., 2022). 구체적으로, 어떤 prime P 에 대해 input $a, b \in \{0, \dots, P-1\}$ 를 받아 합 $c \bmod P$ 를 예측하는 modular addition을 다룬다. 이 task에서 weight decay로 학습한 작은 transformer는 일관되게 grokking을 보인다(Figure 2, Appendix C.2).

주: interactive figure와 재현 코드가 포함된 Colab 자료는 원문의 <https://neelnanda.io/grokking-paper> 에 제공된다.

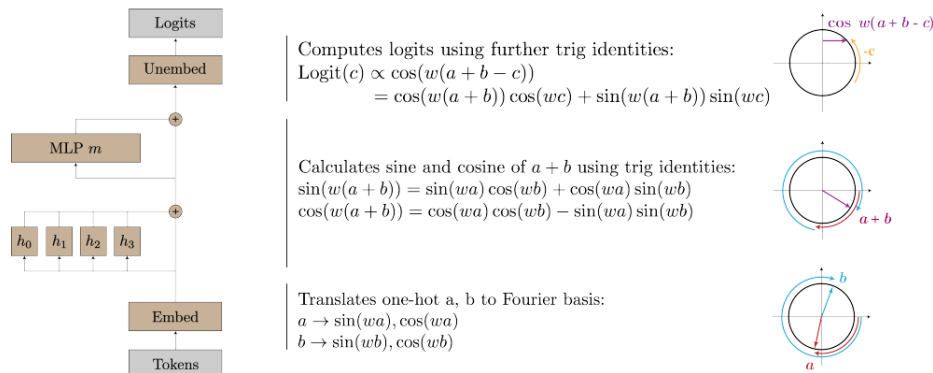


그림 1. modular addition을 위해 one-layer transformer가 구현한 algorithm. 두 수 a, b 가 주어지면 model은 embedding matrix로 각 점을 대응하는 회전에 project한다. attention 및 MLP layer로 이 회전을 compose하여 $a+b \bmod P$ 의 representation을 만든다. 마지막으로 각 $c \in \{0, 1, \dots, P-1\}$ 의 logit을, $-c$ 만큼 회전하여 $\cos(w(a+b-c))$ 를 얻는 방식으로 “read off”한다. w 가 $2\pi/P$ 의 배수이므로 이 값은 $a+b \equiv c \bmod P$ 일 때 최대가 된다.

우리는 이 transformer의 weight를 reverse engineer하여, input을 원으로 map한 뒤 원 위에서 덧셈을 수행한다는 점을 발견했다. 구체적으로 embedding matrix는 input a, b 를 드문 key frequency w_k 집합에서의

sine과 cosine으로 map한다. attention과 MLP layer는 trigonometric identity를 이용해 이를 결합하여 $w_k(a+b)$ 의 sine과 cosine을 계산하고, output matrix는 이 frequency들을 shift하고 결합한다.

다음 네 가지 증거로 이 해석을 확인한다(Section 4). (1) network weight와 activation은 우연으로 보기 어려운 일관된 periodic structure를 보인다. (2) neuron-logit map W_L 은 key frequency의 sinusoidal function 함으로 잘 근사되며, MLP activation을 이 sinusoid에 project하면 neuron에서 trigonometric identity를 “read off”할 수 있다. (3) attention head와 MLP neuron은 단일 frequency의 trigonometric function에 대한 degree-2 polynomial으로 잘 근사된다. (4) model이 사용하는 key frequency를 ablate하면 성능이 chance 수준으로 떨어지는 반면, 나머지 95%의 frequency를 ablate하면 성능이 약간 개선된다.

학습된 algorithm에 대한 이 이해로부터 modular addition task의 두 progress measure를 구성한다. 하나는 non-key frequency를 모두 ablate하는 restricted loss이고, 다른 하나는 모든 key frequency를 ablate하는 excluded loss이다. 두 metric 모두 grokking이 일어나기 전에 연속적으로 개선된다. 이를 이용해 grokking의 training dynamics를 분석한 결과, training은 training data의 **memorization**, generalize하는 mechanism을 학습하는 **circuit formation**, 그리고 weight decay가 memorization component를 제거하는 **cleanup**의 세 단계로 나뉜다. 놀랍게도 grokking에서 perfect test accuracy로의 급격한 전환은 generalizing mechanism을 학습한 뒤 cleanup 중에 일어난다. 따라서 grokking은 sudden shift라기보다 구조화된 mechanism의 점진적 증폭 뒤 memorizing component가 제거되는 현상이다.

2. RELATED WORK

Phase Changes

최근 연구들은 Neural network가 scale-up되거나 더 오래 training될 때 새로운 질적 행동을 빠르게 획득함을 관찰했다(Ganguli et al., 2022; Wei et al., 2022a). McGrath et al. (2021)은 AlphaZero가 10k~30k training step 사이에 인간 chess concept를 다수 빠르게 습득하고, 25k~60k step 사이에 인간 opening theory를 재발명을 발견했다.

Grokking

Grokking은 Power et al. (2022)이 처음 보고했다. 이들은 여러 algorithmic task에서 two-layer transformer를 학습시켰고, perfect train accuracy를 달성한 한참 뒤에 test accuracy가 급격히 증가하는 경우가 많음을 발견했다. Millidge (2022)는 이것이 optimal manifold 위에서의 SGD random walk 때문일 수 있다고 제안했다. 본 연구의 결과는 network가 generalizing algorithm을 향해 연속적으로 진전한다는 점에서 Barak et al. (2022)과 맥을 같이한다. Liu et al. (2022)은 grokking의 작은 예시를 구성하고 이를 사용해 네 개의 learning “phase”가 있는 phase diagram을 계산했다. Thilak et al. (2022)은 explicit regularization이 없어도 slingshot mechanism이라 부른 optimization anomaly로 grokking이 발생할 수 있으며, 이것이 implicit regularizer로 작용할 수 있다고 주장했다.

Circuits-style mechanistic interpretability

Section 4의 post-hoc mechanistic interpretability 방식은 Cammarata et al. (2020), Elhage et al. (2021), Olsson et al. (2022)의 Circuits approach에서 큰 영감을 받았다.

Progress measures

Barak et al. (2022)은 smooth하게 개선되고 emergent behavior보다 앞서는 metric인 progress measure 개념을 도입했다. 이들은 training이 특정 mechanism을 증폭함을 이론적으로 보이고, heuristic하게 progress measure를 정의했다. 반면 본 연구에서는 mechanistic interpretability를 사용하여 empirical하게 progress measure를 발견한다.

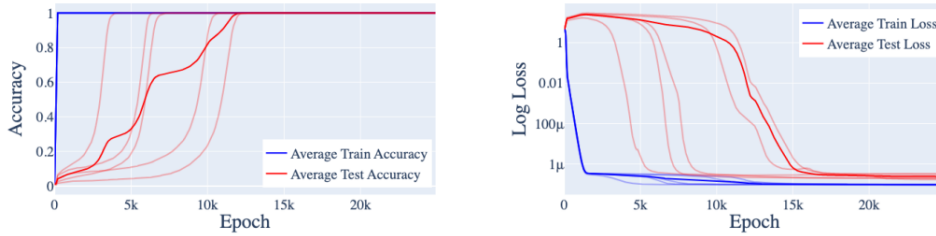


그림 2. Section 3의 modular addition task에서 one-layer transformer를 5개의 random seed로 학습했을 때의 train/test accuracy(왼쪽)와 train/test loss(오른쪽). 이 model은 일관되게 grokking을 보인다. training 초기에 빠르게 overfit한 뒤, 이후 generalize하는 법을 학습한다.

3. SETUP AND BACKGROUND

우리는 transformer가 addition mod P 를 수행하도록 학습한다. model input은 “ $a \ b =$ ”의 형식이며, a 와 b 는 P -dimensional one-hot vector로 encode되고 $=$ 는 그 위에서 output c 를 read하는 special token이다. mainline experiment에서는 $P=113$ 으로 두고, $d=128$ 인 token embedding, learned positional embedding, 차원 $d/4=32$ 의 attention head 4개, MLP의 hidden unit $n=512$ 를 갖는 one-layer ReLU transformer를 사용한다. 다른 experiment에서는 model depth와 dimension을 바꾼다. LayerNorm은 사용하지 않았고 embed/unembed matrix도 tie하지 않았다.

mainline dataset은 가능한 전체 input의 30%, 즉 $113 \cdot 113$ 개 mod P number pair 중 30%로 구성된다. learning rate $\gamma=0.001$, weight decay parameter $\lambda=1$ 인 AdamW optimizer(Loshchilov & Hutter, 2017)로 full-batch gradient descent를 사용한다. 40,000 epoch 동안 training하고, 가능한 pair가 $113 \cdot 113$ 개뿐이므로 training에 쓰지 않은 모든 input pair에서 test loss와 accuracy를 평가한다.

이 task에서 학습된 network는 일관되게 grokking을 보인다. Figure 2에서 보듯 network는 먼저 training set을 overfit한다. train accuracy는 빠르게 100%로 수렴하고 train loss는 빨리 감소하지만 test accuracy는 낮고 test loss는 높게 유지된다. 약 10,000 epoch 뒤 network는 generalize하고 test accuracy는 거의 100%까지 오른다. robustness experiment에서는 다른 architecture와 prime modulus에서도 grokking이 일관되게 발생함을 확인한다(Appendix C.2). Section 5.3에서는 regularization 없이는 grokking이 일어나지 않음을 보인다.

transformer component의 기술에는 Elhage et al. (2021)의 convention과 notation을 따른다. $d \times p$ embedding matrix W_E , MLP layer의 $d \times n$ output matrix W_{out} , 그리고 $P \times d$ unembedding matrix W_U 에 초점을 둔다. $\text{Logits}(a,b)$ 를 input a,b 의 logit vector, $\text{MLP}(a,b)$ 를 MLP activation이라 하자. empirical하게 network는 MLP 주위의 skip connection을 크게 사용하지 않으므로(Appendix A.1), $\text{Logits}(a,b) \approx W_U W_{out} \text{MLP}(a,b)$ 이다. 따라서 $P \times n$ neuron-logit map $W_L = W_U W_{out}$ 도 연구한다.

3.1. THE FOURIER MULTIPLICATION ALGORITHM

학습된 network가 다음 algorithm을 사용한다고 주장한다(Figure 1).

- 두 one-hot encoded token a,b 를 받아, embedding matrix를 통해 여러 frequency $w_k = 2k\pi/P$, $k \in \mathbb{N}$ 에 대해 $\sin(w_k a)$, $\cos(w_k a)$, $\sin(w_k b)$, $\cos(w_k b)$ 로 map한다.
- trigonometric identity를 이용하여 $\cos(w_k(a+b))$ 와 $\sin(w_k(a+b))$ 를 계산한다. 우리 network에서는 attention과 MLP layer가 이 계산을 수행한다.

$$\begin{aligned}\cos(w_k(a+b)) &= \cos(w_k a) \cos(w_k b) - \sin(w_k a) \sin(w_k b) \\ \sin(w_k(a+b)) &= \sin(w_k a) \cos(w_k b) + \cos(w_k a) \sin(w_k b)\end{aligned}$$

- 각 output logit c 에 대해 trigonometric identity로 $\cos(w_k(a+b-c))$ 를 계산한다. 이는 이미 계산한 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ 의 linear function이며 output과 unembedding matrix의 product W_L 에서 구현된다.

$$\cos(w_k(a+b-c)) = \cos(w_k(a+b)) \cos(w_k c) + \sin(w_k(a+b)) \sin(w_k c).$$

(1)

- unembedding matrix는 서로 다른 k 의 $\cos(w_k(a+b-c))$ 도 합한다. 그러면 $c^* = a+b \bmod p$ 에서 cosine wave가 constructive interference를 일으켜 c^* 에 큰 logit을 주고, 나머지 위치에서는 destructive interference를 일으켜 다른 c 에 작은 logit을 준다.

우리는 이 algorithm을 Fourier multiplication이라 부르며, Section 4에서 이 주장을 자세히 정당화한다.

4. REVERSE ENGINEERING A ONE-LAYER TRANSFORMER

이 절에서는 transformer가 Section 3.1의 Fourier multiplication algorithm을 사용한다는 네 가지 증거를 설명한다. Section 3의 mainline model에 분석을 적용하지만, layer 수, training data 비율, prime modulus가 다른 model에서도 결과는 대체로 일관된다(Appendix C.2, 특히 Table 5).

첫 번째 증거는 network weight와 activation을 조사하여 우연으로 나타나기 어려운 일관된 periodic structure를 관찰하는 것이다(Section 4.1). 또한 Fourier transform를 취하면 많은 component가 Fourier domain에서 sparse 또는 거의 sparse하며, 소수의 key frequency에만 support를 둔다.

다음으로 model weight가 실제 구현하는 mechanism을 살핀다(Section 4.2). unembedding matrix W_L 이 (근사적으로) rank 10이며, 각 direction은 5개 key frequency 중 하나의 cosine 또는 sine에 대응함을 보인다. MLP activation을 W_L 의 component에 project하면 대략 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ 의 배수가 생성되므로, MLP layer가 실제로 이 합을 계산함을 알 수 있다.

mechanism을 더 이해하기 위해 individual neuron을 확대해 본다(Section 4.3). attention head와 대부분 neuron은 단일 frequency에서 sine과 cosine의 degree-2 polynomial으로 잘 근사된다. 게다가 W_L 의 대응 direction도 해당 frequency만 포함한다. 이는 model의 computation이 (1) frequency별로 localized되어 있고, (2) 대부분 neuron basis에 aligned되어 있음을 시사한다.

마지막으로 ablation으로 해석의 faithful함을 확인한다(Section 4.4). model의 여러 component를 Fourier multiplication algorithm의 component로 대체했을 때, 성능은 일관되게 저하되지 않았으며 때로는 개선되었다.

4.1. SUGGESTIVE EVIDENCE: SURPRISING PERIODICITY

network가 Section 3.1의 algorithm을 사용한다는 첫 번째 증거는 transformer activation에서의 놀라운 periodicity다. 즉, network 각 부분의 output이 input token의 function으로서 periodic하다.

embedding의 periodicity

먼저 embedding을 분석한다. embedding matrix W_E 의 input dimension을 따라 Fourier transform를 취한 뒤, 다른 dimension을 따라 ℓ_2 -norm을 계산한다(Figure 3). frequency k 와 $P-k$ 의 component norm이 symmetric하므로 앞 56개 frequency만 plot한다. W_E 는 Fourier basis에서 sparse하며, 6개 frequency에서만 유의미하게 0이 아닌 norm을 갖는다. 이 중 이후 model 부분에서 유의미하게 쓰이는 것은 $k \in \{14, 35, 41, 42, 52\}$ 에 대응하는 5개뿐이다. 이를 model의 key frequencies라고 부른다.

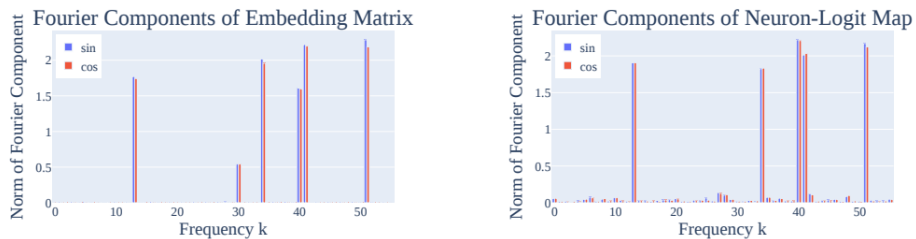


그림 3. (왼쪽) embedding matrix W_E 의 Fourier component norm. Section 4.1에서 논의하듯 Fourier basis에서 W_E 가 sparse하다는 사실은 network가 이 basis에서 작동한다는 증거다. 0이 아닌 6개 frequency 중 5개 “key frequency”가 network의 이후 부분에 나타나며, $k \in \{14, 35, 41, 42, 52\}$ 에 대응한다. (오른쪽) neuron-logit map W_L 의 Fourier component norm. logit axis에 Fourier transform를 취하고 neuron axis에 norm을 취했다. Section 4.2에서 논의하듯 W_L 은 5개 key frequency w_k 로 잘 근사된다.

attention head와 MLP neuron activation의 periodicity

이 periodic structure는 network 전체에서 되풀이된다. 예로 Figure 4는 head 0에서 두 input의 모든 조합에 대해 position 0의 attention weight를 plot한다. attention은 frequency $k=35$ 의 periodic structure를 보인다. Figure 4에는 input 조합마다 MLP neuron 0의 activation도 표시했다. activation은 $k=42$ 의 periodicity를 갖는다. 다른 attention head와 MLP neuron에서도 유사한 pattern을 본다(Appendix C.1).

logit의 periodicity

마지막으로 logit도 periodic하다. Figure 4에서는 input a, b 에 대한 2D Fourier basis에서 logit을 표현한 뒤 output dimension에 대해 ℓ_2 -norm을 취한다. 5개 key frequency w_k 의 sine과 cosine product에 대응하는 20개 component만 유의미한 norm을 가지며, Figure 4에서는 5개의 2×2 block으로 나타난다.

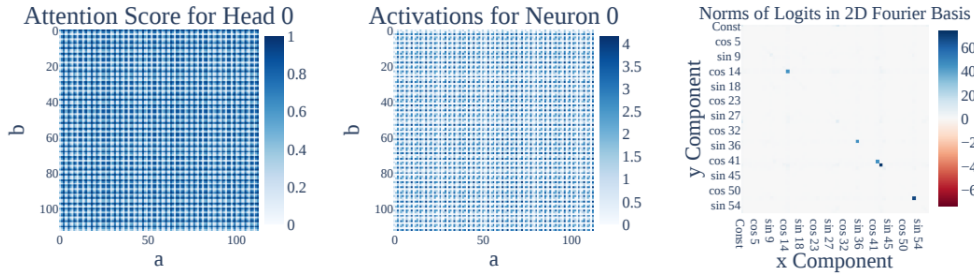


그림 4. (왼쪽) token '='에서 'a'로 향하는 head 0의 attention score를 input a, b 의 function으로 표시했다. (가운데) input a, b 에서 MLP neuron 0의 activation. attention score와 neuron activation 모두 periodic하다(Section 4.1). (오른쪽) logit Fourier component의 norm. input a, b 에 대해 2D Fourier transform를 취하고 logit axis에 norm을 취했다. 5개 key frequency에 대응하는 20개 유의미 component가 있다.

4.2. MECHANISTIC EVIDENCE: COMPOSING MODEL WEIGHTS

이제 model이 trigonometric identity (1)을 다음과 같이 구현함을 보인다. 즉, $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ function은 MLP activation에 linear하게 representation되고, unembed matrix는 이 linear direction을 read하여 각각 $\cos(w_k c)$, $\sin(w_k c)$ 와 곱한다.

두 단계로 보인다. 먼저 MLP activation에서 logit으로 map하는 matrix W_L 이 (근사적으로) rank 10이며 다음처럼 근사됨을 보인다.

$$W_L = \sum_{k \in \{14, 35, 41, 42, 52\}} \cos(w_k) u_k^T + \sin(w_k) v_k^T \quad (2)$$

여기서 $u_k, v_k \in \mathbb{R}^{512}$ 이고, $\cos(w_k)$, $\sin(w_k) \in \mathbb{R}^{113}$ 는 c 번째 entry가 각각 $\cos(w_k c)$, $\sin(w_k c)$ 인 vector다. 둘째, model은 input a, b 의 logit을 다음과 같이 구현한다.

$$\text{Logits}(a, b) = W_L \text{MLP}(a, b) \approx \sum_k \cos(w_k) u_k^T \text{MLP}(a, b) + \sin(w_k) v_k^T \text{MLP}(a, b). \quad (3)$$

우리는 $u_k^T \text{MLP}(a,b)$, $v_k^T \text{MLP}(a,b)$ 항이 각각 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ 의 근사 배수임을 empirical하게 확인한다(variance explained 90% 초과). 따라서 model은 주장한 대로 MLP에서 trigonometric function을 계산하고 이를 read off한다. sanity check로 logit이 실제로 $\cos(w_k(a+b-c))$ 꼴의 항으로 잘 근사됨도 확인한다(variance explained 95%).

W_L component	$u_k^T \text{MLP}(a,b)$ 또는 $v_k^T \text{MLP}(a,b)$ 의 Fourier component	FVE
$\cos(w_{14}c)$	$44.6 \cos(w_{14}a)\cos(w_{14}b) - 43.6 \sin(w_{14}a)\sin(w_{14}b) \approx 44.1 \cos(w_{14}(a+b))$	93.2%
$\sin(w_{14}c)$	$44.1 \sin(w_{14}a)\cos(w_{14}b) + 44.1 \cos(w_{14}a)\sin(w_{14}b) \approx 44.1 \sin(w_{14}(a+b))$	93.5%
$\cos(w_{35}c)$	$40.7 \cos(w_{35}a)\cos(w_{35}b) - 43.6 \sin(w_{35}a)\sin(w_{35}b) \approx 42.2 \cos(w_{35}(a+b))$	96.8%
$\sin(w_{35}c)$	$41.8 \sin(w_{35}a)\cos(w_{35}b) + 41.8 \cos(w_{35}a)\sin(w_{35}b) \approx 41.8 \sin(w_{35}(a+b))$	96.5%
$\cos(w_{41}c)$	$44.8 \cos(w_{41}a)\cos(w_{41}b) - 44.8 \sin(w_{41}a)\sin(w_{41}b) \approx 44.8 \cos(w_{41}(a+b))$	97.0%
$\sin(w_{41}c)$	$44.5 \sin(w_{41}a)\cos(w_{41}b) + 44.5 \cos(w_{41}a)\sin(w_{41}b) \approx 44.5 \sin(w_{41}(a+b))$	97.0%
$\cos(w_{42}c)$	$64.6 \cos(w_{42}a)\cos(w_{42}b) - 68.5 \sin(w_{42}a)\sin(w_{42}b) \approx 66.6 \cos(w_{42}(a+b))$	96.4%
$\sin(w_{42}c)$	$67.8 \sin(w_{42}a)\cos(w_{42}b) + 67.8 \cos(w_{42}a)\sin(w_{42}b) \approx 67.8 \sin(w_{42}(a+b))$	96.4%
$\cos(w_{52}c)$	$60.5 \cos(w_{52}a)\cos(w_{52}b) - 65.5 \sin(w_{52}a)\sin(w_{52}b) \approx 63.0 \cos(w_{52}(a+b))$	97.4%
$\sin(w_{52}c)$	$64.5 \sin(w_{52}a)\cos(w_{52}b) + 64.5 \cos(w_{52}a)\sin(w_{52}b) \approx 64.5 \sin(w_{52}(a+b))$	98.2%

표 1. unembedding matrix에서 $\cos(w_k)$, $\sin(w_k)$ component에 각각 대응하는 direction u_k , v_k 에 대해 MLP activation과 dot product를 취한 뒤 Fourier transform을 수행했다(가운데 열에는 최대 coefficient 두 개만 표시). projection을 $\cos(w_k(a+b))$ 또는 $\sin(w_k(a+b))$ 에 비례하는 단일 항으로 대체한 경우의 FVE는 일관되게 1에 가깝다.

W_L 은 $\cos(w_k c)$, $\sin(w_k c)$ 로 잘 근사된다. W_L 의 logit axis에 DFT를 수행하고 sine 및 cosine에 대응하는 10개 direction u_k, v_k 를 본다. 식 (2)의 합으로 W_L 을 근사하면 residual의 Frobenius norm은 W_L norm의 0.55% 미만이다. 즉 W_L 은 각 key frequency의 cosine과 sine에 대응하는 10개 direction으로 잘 근사된다. Figure 3의 direction norm도, 5개 key frequency 밖에는 유의미한 Fourier component가 없음을 보여 준다.

unembedding matrix는 MLP neuron에서 $\cos(w_k(a+b))$ 및 $\sin(w_k(a+b))$ 꼴의 항을 read off한다. 다음으로 $k \in \{14, 35, 41, 42, 52\}$ 의 각 u_k, v_k 와 MLP activation의 dot product를 취한다. Table 1이 보이듯 이 dot product는 다음 항의 배수로 잘 근사된다.

$$\begin{aligned}\cos(w_k(a+b)) &= \cos(w_k a)\cos(w_k b) - \sin(w_k a)\sin(w_k b) \\ \sin(w_k(a+b)) &= \sin(w_k a)\cos(w_k b) + \cos(w_k a)\sin(w_k b)\end{aligned}$$

즉 각 key frequency k 에 대해 u_k, v_k 는 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ 를 representation하는 MLP neuron activation space의 linear direction이다.

logit은 $\cos(w_k(a+b-c))$ 의 weighted sum으로 잘 근사된다. output logit을 $k \in \{14, 35, 41, 42, 52\}$ 에 대해 $\sum_k \alpha_k \cos(w_k(a+b-c))$ 로 근사하고 ordinary least squares로 coefficient α_k 를 fit한다. 이 근사는 원래 logit variance의 95%를 설명한다. output logit은 113·113·113-dimensional vector인데도 이 해석이 예측한 단 5개 direction으로 잘 근사된다는 점은 놀랍다. 이 logit approximation을 사용하여 test loss를 평가하면 loss는 실제로 $2.4 \cdot 10^{-7}$ 에서 $4.7 \cdot 10^{-8}$ 로 개선된다. 종합하면 model은 다음 꼴의 항을 합해 계산한다.

$$\cos(w_k(a+b-c)) = \cos(w_k(a+b))\cos(w_k c) + \sin(w_k(a+b))\sin(w_k c).$$

4.3. ZOOMING IN: APPROXIMATING NEURONS WITH SINES AND COSINES

앞 절에서는 W_L 을 사용해 MLP neuron에 representation된 trigonometric identity를 read off함으로써 model이 final logit을 계산하는 방식을 보였다. 이제 identity가 MLP layer에 어떻게 representation되는지 이해하기 위해 attention head와 MLP neuron을 살핀다. Appendix C.1.2에서는 attention head 두 개가 특정 frequency의 sine과 cosine에 대한 degree-2 polynomial을 대략 계산을 보인다(나머지 두 head는 residual stream에서 input embedding 크기를 키우는 데 사용된다). 여기서는 대부분 neuron도 degree-2 polynomial로 잘 근사되고, neuron에서 logit으로의 map이 frequency별로 localized됨을 보인다.

대부분 MLP neuron은 단일 frequency의 degree-2 polynomial을 대략 계산한다. 각 MLP neuron의 activation을 5개 key frequency 중 하나의 degree-2 polynomial로 근사해 보았다. Figure 5에서 보듯 전체 512개 neuron 중 433개(84.6%)는 단일 frequency로 variance의 85% 이상이 설명된다.

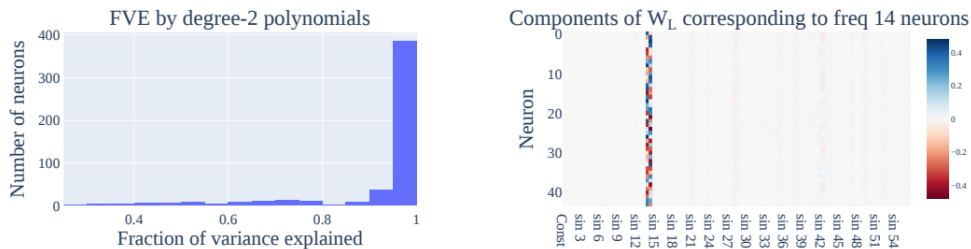


그림 5. (왼쪽) 대부분 neuron은 단일 frequency의 degree-2 polynomial으로 잘 근사된다. (오른쪽) frequency 14 neuron 44개 각각에 대응하는 W_L weight를 보이는 heatmap. 0이 아닌 component는 $k=14$ 에서 $\sin(w_k)$, $\cos(w_k)$ 에 대응한다.

logit으로의 map은 frequency별로 localized된다. variance explained가 가장 큰 frequency에 따라 위 433개 neuron을 나눈다. 각 subset에서 neuron에서 logit으로의 map W_L 은 해당 frequency의 sine과 cosine에 대응하는 두 개의 0이 아닌 component만 갖는다. 예를 들어 Figure 5는 $k=14$ cluster의 neuron 44개에 대응하는 W_L 44개 column을 보여주며, 유의미한 component는 $k=14$ 의 $\sin(2k\pi/P)$, $\cos(2k\pi/P)$ 뿐이다.

4.4. CORRECTNESS CHECKS: ABLATIONS

앞 절에서 model의 여러 component가 sine과 cosine의 sparse combination으로 잘 근사됨을 보였다. 이제 각 component를 그 approximation으로 교체하여, 이 근사가 model 기능에 faithful한지 확인한다. 일반적으로 성능을 해치지 않으며, 어떤 경우에는 개선한다.

MLP neuron. Section 4.3에서 degree-2 polynomial로 잘 근사된 neuron 433개를 찾았다. 각 neuron의 activation value를 대응 polynomial로 대체하고 나머지 neuron은 그대로 둔다. 그러면 loss는 상대적으로 3%만 증가한다($2.41 \cdot 10^{-7}$ 에서 $2.48 \cdot 10^{-7}$ 으로). accuracy에는 영향이 없다. 더 엄격한 ablation으로 각 neuron activation을 key frequency에서 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ 꼴에 대응하는 polynomial component로만 제한할 수도 있다. 이때 loss가 77% 개선되어 $5.54 \cdot 10^{-8}$ 이 되며, Section 4.2의 neuron trigonometric identity로 logit을 계산한다는 설명을 뒷받침한다.

Logit frequency. 이어 final logit의 여러 component를 Fourier space에서 ablate한다. 모든 113·113 input pair에 대한 113·113·113 logit matrix를 input 2D DFT하여 Fourier basis logit을 얻고, 이 basis의 여러 frequency를 0으로 설정한다. 먼저 각 key frequency에 대응하는 component를 ablate한다. Figure 6처럼 어느 key frequency 하나를 ablate해도 loss가 크게 증가한다. 따라서 앞 절에서 찾은 5개 frequency가 transformer의 필요 component임을 확인한다. 반대로 다른 frequency의 ablation은 model을 전혀 해치지 않는다. 이후 key frequency 외의 Fourier component 113·113-40개를 모두 ablate하면, 오히려 성능이 개선되어 loss가 70% 하락해 $7.24 \cdot 10^{-8}$ 이 된다.

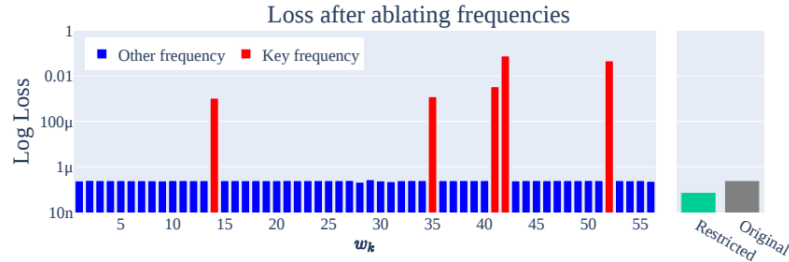


그림 6. 각 frequency $k \in \{1, 2, \dots, 56\}$ 와 5개 key frequency를 제외한 모든 frequency를 ablate했을 때(restricted loss) transformer loss(낮을수록 좋음). 참고를 위해 original, ablation하지 않은 loss도 넣었다. key frequency를 ablate하면 성능이 떨어지지만, 나머지 frequency의 ablation은 성능을 해치지 않는다.

5. UNDERSTANDING GROKING BEHAVIOR USING PROGRESS MEASURES

이제 network에 대한 mechanistic understanding으로부터, phase transition 중을 포함한 training 과정에서 model progress를 추적할 수 있는 두 progress measure를 정의한다. 이를 통해 network가 final solution에 이르는 방식을 연구할 수 있다.

5.1. PROGRESS MEASURES

Section 4.4의 ablation을 restricted loss와 excluded loss라는 두 progress measure로 바꾼다.

Restricted loss. final network는 sparse한 frequency 집합 w_k 를 사용하므로, intermediate model이 그 frequency만으로 얼마나 잘 수행하는지 확인하는 것이 자연스럽다. 이를 위해 logit에 2D DFT를 수행하여 a, b wave의 linear combination으로 쓰고, constant term과 5개 key frequency에 대해 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ 에 대응하는 20개 term을 제외한 나머지를 0으로 둔다. 그런 뒤 ablated network loss를 측정한다.

Excluded loss. 중요한 frequency w_k 를 유지하는 대신, 이제 logit에서 해당 key frequency만 제거하고 나머지는 보존한다. Fourier multiplication과 memorization이 성능에 각각 얼마나 기여하는지 추적하기 위해 이를 training data에서 측정한다. memorizing solution은 Fourier domain에 널리 퍼져 있어서 소수 direction을 ablate해도 대부분 영향받지 않는 반면, generalizing solution은 크게 손상될 것이라는 생각이다.

이 밖에 (1) W_E, W_L Fourier component norm의 Gini coefficient(Hurley & Rickard, 2009)를 측정한다. 이는 Fourier basis에서 W_E, W_L 의 sparsity를 잴다. 또한 (2) train loss가 거의 0이 되면 weight decay가 낮출 것으로 기대되는 training 중 weight의 ℓ_2 -norm을 측정한다.

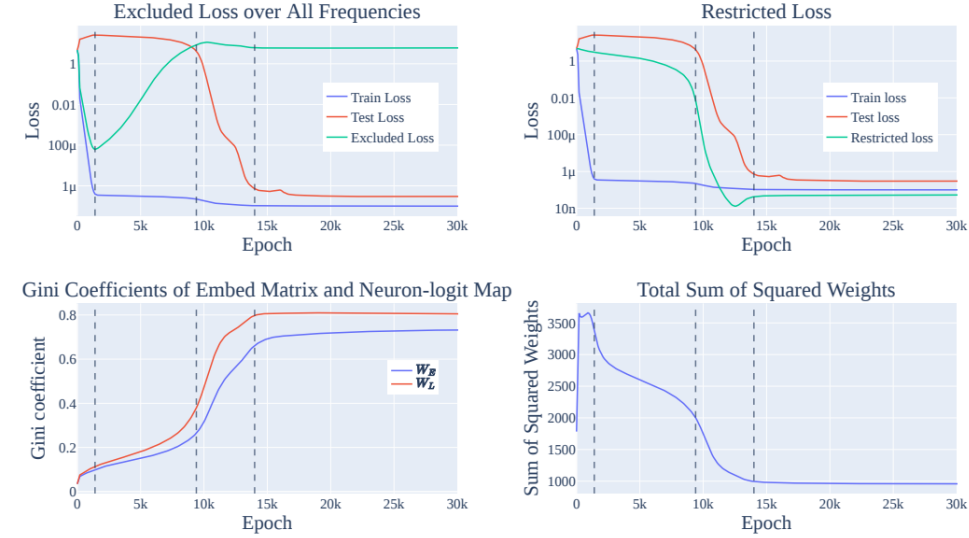


그림 7. Section 5.1 progress measure가 training 과정에서 변하는 방식. 선은 memorization, circuit formation, cleanup(그리고 마지막 stable phase)의 3개 training phase를 구분한다. (왼쪽 위) circuit formation 동안 excluded loss는 증가하지만 train/test loss는 평평하게 유지된다. (오른쪽 위) restricted loss는 test loss보다 먼저 감소하기 시작하나, grokking이 시작될 때 inflection point가 있다. (왼쪽 아래) W_E, W_L Fourier component norm의 Gini coefficient는 cleanup 동안 급격히 증가한다. (오른쪽 아래) squared weight의 합은 circuit formation 중 smooth하게, cleanup 중 더 급하게 감소하므로 두 phase 모두 weight decay와 연결된다.

W_L 의 direction. Section 4.2에서 W_L 이 key frequency의 cosine과 sine에 대응하는 10개 direction으로 잘 근사됨을 보였다. MLP activation을 이 10개 direction에 project하면 loss는 50% 낮아져 $1.19 \cdot 10^{-7}$ 가 된다. 대신 이 10 direction의 nullspace에 MLP activation을 project하면 loss는 5.27로 증가하여 uniform보다도 나빠진다. 이는 network가 이 10개 direction만을 사용하여 low loss를 달성함을 시사한다.

5.2. PHASES OF GROKKING: MEMORIZATION, CIRCUIT FORMATION, AND CLEANUP

Section 4의 mainline model에서 excluded loss, restricted loss, matrix W_U 및 W_L 의 Gini coefficient, squared weight 합을 Figure 7에 표시한다. training은 memorization, circuit formation, cleanup이라는 세 phase로 나뉜다(Appendix C.2는 다른 model에서도 유사한 결과를 보인다).

Memorization (Epoch 0k–1.4k). excluded loss와 train loss가 모두 감소하지만 test loss와 restricted loss는 높게 남고 Gini coefficient도 비교적 평평하다. 즉 model은 data를 memorize하며 final model이 쓸 frequency w_k 는 사용되지 않는다.

Circuit formation (Epoch 1.4k–9.4k). excluded loss는 상승하고 squared weight 합은 하락하며 restricted loss는 하락하기 시작하지만 test와 train loss는 평평하다. 이는 train set에서 model 행동이 memorizing solution에서 Fourier multiplication algorithm으로 smooth하게 전환됨을 시사한다. squared weight 합의 하락은 circuit formation이 weight decay 때문에 일어날 가능성을 시사한다. 특히 circuit은 grokking이 일어나기 훨씬 전에 형성된다.

Cleanup (Epoch 9.4k–14k). excluded loss는 plateau에 이르고 restricted loss는 계속 떨어지며 test loss는 갑자기 하락하고 squared weight 합도 급격히 하락한다. 완성된 Fourier multiplication circuit은 task를 잘 풀고 memorization circuit보다 weight가 작으므로, weight decay는 network가 memorized solution을 버리고 Fourier multiplication circuit에 집중하도록 만든다. 이는 W_E , W_L 의 Gini coefficient가 급격히 증가하는 현상에서 가장 명확하다. 이는 network가 Fourier basis에서 더 sparse해지고 있음을 뜻한다.

5.3. GROKING AND WEIGHT DECAY

앞 절에서 grokking의 각 phase가 weight의 ℓ_2 -norm inflection point에 대응함을 보았다. 이는 weight decay가 grokking의 중요한 component이며 generalizing solution을 향한 progress를 이끈다는 점을 시사한다. Appendix D.1은 weight decay가 grokking에 필요하다는 추가 증거를 제시한다. weight decay가 작을수록 network가 grok하는 데 훨씬 오래 걸리며(Liu et al. (2022)의 toy model 결과와도 일치), weight decay나 다른 regularization이 없으면 modular arithmetic task에서 network가 grok하지 않는다. Appendix C.2에서는 data 양도 grokking에 영향을 준다. 충분한 data가 주어지면 train/test loss 사이의 gap이 사라지고, 두 loss가 training 몇 epoch 뒤 함께 급격히 감소한다. Appendix D.3에서는 몇 가지 추가 algorithmic task에서도 이 결과를 재현한다.

6. CONCLUSION AND DISCUSSION

본 연구는 modular addition task에서 학습된 작은 transformer를 대상으로 mechanistic interpretability를 사용해 progress measure를 정의했다. transformer는 input을 R^2 의 회전에 embed하고, trigonometric identity로 회전을 compose하여 $a+b \bmod 113$ 을 계산한다. reverse-engineered algorithm을 사용해 두 progress measure를 정의했으며, network는 grokking phase change 전에 이 measure를 따라 final algorithm을 향해 연속적으로 progress한다. 본 연구는 emergent behavior를 이해하기 위해 mechanistic interpretability를 사용하는 proof of concept로 볼 수 있다.

Larger model과 realistic task

본 연구는 단일 circuit으로 해결되는 단순 algorithmic task에서 small transformer의 행동을 연구했다. 반면 larger model은 훨씬 어려운 task를 해결하기 위해 더 크고 더 많은 circuit을 사용한다(Cammarata et al., 2020; Wang et al., 2022). 여기 보고한 분석에는 상당한 manual effort가 필요했고 progress metric도 하나의

특정 algorithmic task의 small network에 특화되어 있다. 다른 larger model로 확장하려면 분석을 자동화하고 task-independent progress measure를 찾는 방법이 필요해 보인다. 더 realistic한 application의 가능한 scenario는 Appendix F에서 논의한다.

Phase change threshold의 발견

Section 5.1에서 정의한 progress measure는 phase transition 전에 비교적 smooth하게 증가하여 이 task의 grokking을 이해하는 데는 충분하지만, phase transition이 정확히 언제 발생할지 사전에 예측할 수 있는 일반적인 criticality 개념은 없다. 향후 연구는 emergent behavior의 timing을 예측하도록 progress measure를 적용할 theory와 practice를 발전시켜야 한다.

REPRODUCIBILITY STATEMENT

model checkpoint download instruction을 포함하여 결과를 재현하는 code가 담긴 annotated Colab notebook은 원문의 <https://neelnanda.io/grokking-paper>에 제공된다.

AUTHOR CONTRIBUTIONS

Neel Nanda는 주된 research contributor였다. mainline model의 weight를 reverse engineer하여 Fourier multiplication algorithm을 발견했고 Section 4의 증거를 찾았다. restricted/excluded loss progress measure와 mainline model grokking을 세 discrete phase로 나눌 수 있다는 사실도 발견했다. 마지막으로 phase transition이 있는 다른 setting에서 grokking을 보임으로써 grokking, limited data, phase transition의 연결을 찾았다.

Lawrence Chan은 이 연구의 framing과 technical writing에 매우 중요한 기여를 했다. 또한 Gini coefficient progress measure를 만들고, mainline model 결과가 다른 random seed, architecture, prime modulus, regularization method를 포함한 다른 small transformer model에 어느 정도 적용되는지 Appendix에서 분석했다.

Tom Lieberum은 LayerNorm이 없는 modular addition task의 1L Transformer로 grokking의 minimal setup을 만들고 model internal에서 놀라운 periodicity를 발견하며 연구 초기 단계에 기여했다.

Jess Smith는 서로 다른 random seed, architecture, 기타 hyper-parameter에서 grokking을 탐색하는 experiment를 수행했다.

Jacob Steinhardt는 결과를 명확히 하고 정제하도록 도왔으며, 상당한 편집 및 writing feedback을 제공했고 progress measure frame을 제안했다.

ACKNOWLEDGMENTS

저자들은 Oliver Balfour, David Bau, Sid Black, Nick Cammarata, Stephen Casper, Bilal Chughtai, Arthur Conmy, Xander Davies, Ben Edelman, Nelson Elhage, Ryan Greenblatt, Jacob Hilton, Evan Hubinger, Zac Kenton, Janos Kramar, Lauro Langosco, Tao Lin, David Lindner, Eric Michaud, Vlad Mikulik, Noa Nabeshima, Chris Olah, Michela Paganini, Alex Ray, Rohin Shah, Buck Shlegeris, Alex Silverstein, Ben Toner, Johannes Treutlein, Nicholas Turner, Vikrant Varma, Kevin Wang, Martin Wattenberg, John Wentworth, Jeff Wu와의 correspondence 및 feedback 덕분에 이 논문의 사고와 서술이 크게 명확해졌다고 밝힌다. Adam Gleave와 Chengcheng Tan의 substantial editing help, 그리고 Neel과의 pair programming을 한 Noa Nabeshima와 Vlad Mikulik에게도 감사한다.

이 연구는 Elhage et al. (2021)과 Olsson et al. (2022)이 개발한 interpretability technique와 framework에 크게 의존했다. model training에는 PyTorch(Paszke et al., 2019)를, data analysis에는 NumPy(Harris et al., 2020), Pandas(Wes McKinney, 2010), einops(Rogozhnikov, 2022)를 사용했다. figure는 Plotly(Plotly Technologies Inc., 2015)로 만들었다.

Neel은 논문 기여 중 개인적 어려움을 헤쳐 나갈 때 practical/emotional support를 준 Jemima Jones와, distillation stage에서 훌륭한 research environment를 제공한 Schelling Residency에 감사를 표한다. 또한 이 조사가 가능해지는 데 결정적으로 기여한, 특히 Chris Olah를 포함한 Anthropic interpretability team의 매우 관대한 mentorship에도 감사를 표한다.

APPENDIX A. TRANSFORMER의 MATHEMATICAL STRUCTURE

model을 기술할 때 Elhage et al. (2021)의 convention과 notation을 따른다. 여기서는 해당 notation을 간략히 되짚고, 본 연구의 specific case에서 살핀다.

hyperparameter는 다음과 같이 표기한다. $d_{\text{vocab}}=113$ 은 input/output space의 크기(=는 별도 취급), $d_{\text{model}}=128$ 은 residual stream의 width(embedding size), $d_{\text{head}}=32$ 는 한 attention head의 query/key/value vector 크기, $d_{\text{mlp}}=512$ 는 neuron 수다.

parameter는 W_E (embedding layer), W_{pos} (positional embedding), attention layer head j 의 네 weight matrix W_Q^j (query), W_K^j (key), W_V^j (value), W_O^j (attention output), MLP layer input linear map의 W_{in} , b_{in} , output linear map의 W_{out} , b_{out} , unembedding layer W_U 로 표기한다. embedding, attention layer, unembedding에는 bias가 없으며, embedding/unembedding layer matrix도 tie하지 않는다.

이제 network의 mathematical structure를 설명한다. loss는 final token의 logit에서만 계산되고 token 사이 정보 이동은 attention layer에서만 일어나므로, attention layer 끝 이후의 variable은 final token만 가리킨다. position i 의 token(one-hot encoded vector)을 t_i , i 번째 positional embedding을 p_i , index i token의 initial residual stream을 $x_i^{(0)}$, head i 에서 $=$ 로부터 모든 previous token으로의 attention score를 $A^{(i)}$, final token의 attention layer 뒤 residual stream을 $x^{(1)}$, final token의 MLP activation을 MLP, final residual stream을 $x^{(2)}$, final token의 logit을 Logits로 둔다. logit은 다음 equation으로 계산된다.

$$\begin{aligned} x_i^{(0)} &= W_E t_i + p_i \\ A^j &= \text{softmax}(x^{(0)T} W_K^j W_Q^j x_2^{(0)}) \\ x^{(1)} &= [\sum_j W_O^j W_V^j (x^{(0)} \cdot A^j)] + x_2^{(0)} \\ \text{MLP} &= \text{ReLU}(W_{\text{in}} x^{(1)}) \\ x^{(2)} &= W_{\text{out}} \text{MLP} + x^{(1)} = W_{\text{out}} \text{ReLU}(W_{\text{in}} x^{(1)}) + x^{(1)} \\ \text{Logits} &= W_U x^{(2)} \end{aligned}$$

Elhage et al. (2021)처럼 $W_O^j W_V^j (x^{(0)})$ 항을 head j 의 OV circuit이라 부른다.

A.1. EMPIRICAL MODEL SIMPLIFICATIONS

두 가지 empirical observation을 한다.

- ‘=’가 자기 자신에게 주는 attention은 trivial하다. 실제로 각 head에서 average attention은 0.1%~0.4%이고, 이를 ablate해도 model performance는 전혀 변하지 않는다.
- MLP layer 주위의 skip connection은 model computation에 중요하지 않아 무시할 수 있다. 구체적으로 이를 0 또는 average로 설정하는 zero/mean ablation을 해도 model accuracy는 바뀌지 않으며

loss는 $2.4 \cdot 10^{-7}$ 에서 각각 $9.12 \cdot 10^{-7}$, $7.25 \cdot 10^{-7}$ 로 바뀐다. loss 증가는 유의미하지만 baseline이 매우 작아 model computation을 reverse engineer할 때는 여전히 무시할 수 있다. 다만 attention head와 그 주위 skip connection은 model function에 결정적이다. attention head zero ablation은 loss를 24.3으로, attention-head skip connection zero ablation은 19.1로 높이며, 둘 다 chance보다 훨씬 나쁘다.

첫 observation의 결과 attention은 2개 element에 대한 softmax, 즉 difference에 대한 sigmoid가 된다. $x_2^{(0)}$ 는 x, y 와 independent하므로 constant이고, '='의 embedding/positional embedding도 fixed다. 따라서

$$A_0^j = \sigma(x_2^{(0)T} W_Q^j W_K^j (x_0^{(0)} - x_1^{(0)})), \quad A_1^j = 1 - A_0^j.$$

둘째 observation의 결과 $\text{Logits} \approx W_U W_{\text{out}} \text{MLP}$ 이고, 이를 $W_L = W_U W_{\text{out}}$ 로 표기한다. network 관점에서는 두 constituent 각각이 아니라 linear하게 compose되는 W_L 이 의미 있는 matrix다.

APPENDIX B. WHY USE CONSTRUCTIVE INTERFERENCE?

Section 4와 Appendix C.2.1에서 보이듯, 이 task에서 학습된 small transformer는 여러 frequency를 더한다. 그 이유는 $x=0 \bmod 113$ 에서의 function value가 다른 어떤 x 보다도 훨씬 크게 되는 function을 만들기 위해서다.

예를 들어 $f_{14}(x) = \cos((2\pi \cdot 14/113)x)$ 를 보자. 이 function은 period 113이고 $x=0 \bmod 113$ 에서 maximum이다. 그러나 다른 값에서도 1에 가깝다. 예컨대 $f_{14}(8) = f_{14}(105) = 0.998$, $f_{14}(16) = f_{14}(89) = 0.994$ 등이다.

이제 $f_{35}(x) = \cos((2\pi \cdot 35/113)x)$ 를 보자. 이 function도 period 113이고 $x=0 \bmod 113$ 에서 maximum이지만, $f_{35}(8) = f_{35}(105) = -0.990$ 이다. 따라서 f_{14} 과 f_{35} 를 더하면 $x=8 \bmod 113$ 에서 1에 가까운 function이 되지 않는다. 마찬가지로 $f_{35}(16) = 0.961$ 인 반면 $f_{52}(16) = -0.56$ 이므로, 세 번째 frequency를 더하면 $x=16 \bmod 113$ 의 peak도 낮아진다. mainline model이 사용한 5개 frequency cosine wave의 constructive interference를 Figure 8에 제시한다.

Constructive Interference of Cosine Waves of Different Frequencies

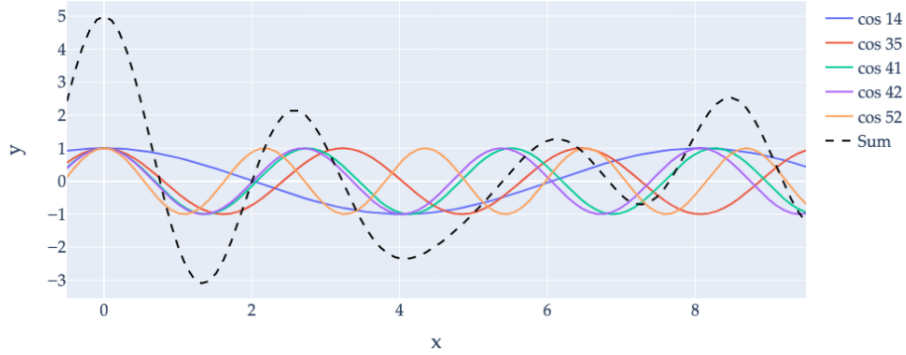


그림 8. Appendix B에서 설명했듯, 모든 $k \in [0, \dots, P-1]$ 에 대해 $\cos((2k\pi/P)x)$ 는 $x=0 \bmod 113$ 에서 maximum value 1을 얻지만, maximum에 가까운 값을 갖는 추가 peak도 있다. 그러나 5개 key frequency의 cosine wave를 더하면 model은 $x=0 \bmod 113$ 에서의 값이 다른 어느 곳보다도 유의미하게 큰 periodic function을 구성한다.

APPENDIX C. MODULAR ARITHMETIC NETWORK의 MECHANISTIC ANALYSIS를 위한 SUPPORTING EVIDENCE

C.1. 본문에서 다룬 특정 training run의 추가 분석

이 절에서는 mainline model과 관련한 추가 evidence를 제공한다.

C.1.1. 다른 attention head activation의 periodicity

Figure 9에는 마지막 token '='에서 첫 token a로 향하는 4개 attention head 모두의 attention pattern을 input a,b에 대한 heatmap으로 표시한다. 각 head에서 이는 scalar다. 눈에 띄는 periodicity가 관찰되며, head 1과 3은 같은 frequency를 representation하는 반면 head 0과 2는 서로 다른 frequency를 representation한다. Appendix A.1에서 보였듯 '='가 자기 자신에게 주는 attention은 무시할 수 있으므로 $A_0^j = 1 - A_1^j$ 이며 a에 대한 attention만 plot하면 충분하다.

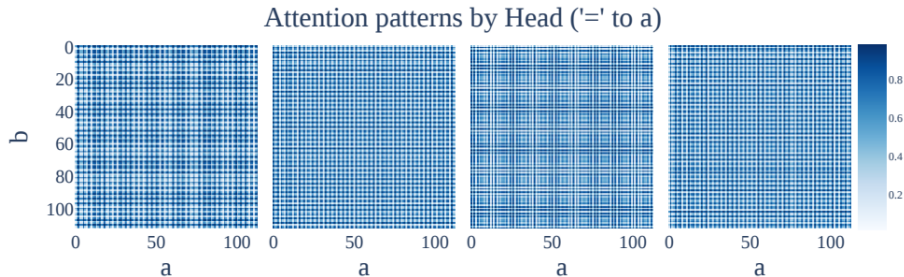


그림 9. 세 번째 sequence position의 '=' token에서 첫 번째 sequence position의 a token으로 향하는 각 head의 attention pattern을 input 위의 heatmap으로 표시했다. 4개 attention head 모두 두드러진 periodicity를 보인다.

Head	k	α_j	β_j	FVE
0	35	-0.26	-0.14	99.03%
1	42	0.27	-0.04	98.49%
2	52	0.29	-0.05	99.07%
3	42	-0.26	0.04	97.91%

표 2. 각 attention head에서 '='에서 a로 향하는 pattern은 $0.5 + \alpha(\cos(w_k a) - \cos(w_k b)) + \beta(\sin(w_k a) - \sin(w_k b))$ 로 잘 근사된다. coefficient와 이 근사의 FVE를 제시한다.

C.1.2. attention head를 sine/cosine으로 근사하기

attention head는 단일 frequency의 degree-2 polynomial을 대략 계산하거나 W_E 를 증폭하는 데 쓰인다. $\cos(w_k(a+b))$ 같은 항을 계산하려면 model은 W_E 가 output하는 sine/cosine embedding의 product를 계산해야 한다. attention head는 attention weight와 OV circuit의 product이므로 대략 bilinear하며, 이 계산에 자연스러운 위치다. 실제로 각 head에서 attention score Fourier transform는 단일 frequency w_k 에 concentrate된다. 4개 head 중 2개는 대응 OV circuit도 같은 frequency에 concentrate된다. 또한 attention score를 attention weight로 map하는 softmax는 대략 linear하게 동작하는 regime에 있으며, 이를 linear function으로 대체하면 성능이 실제로 개선된다. 따라서 attention weight가 OV output과 곱해져 cosine/sine addition formula에 필요한 w_k 의 degree-2 polynomial을 만든다.

남은 2개 head는 attention score의 합이 대략 1이고 OV circuit은 5개 key frequency를 모두 포함한다. 이는 residual stream에서 key frequency의 magnitude를 키우는 데 사용됨을 시사한다. 이 주장은 Appendix C.1.3에서 확인한다.

C.1.3. attention pattern weight는 단일 frequency의 sine/cosine difference로 잘 근사된다

attention head periodicity는 매우 특징적인 꼴을 갖는다. 즉 일부 frequency w_k 와 head별로 달라질 수 있는 constant α_j, β_j 에 대해 A_0^j 는 $0.5 + \alpha_j(\cos(w_k a) - \cos(w_k b)) + \beta_j(\sin(w_k a) - \sin(w_k b))$ 로 잘 근사된다. 이는 어떤 constant γ, θ 에 대해 $0.5 + \gamma(\cos(w_k(a+\theta)) - \cos(w_k(b+\theta)))$ 로 단순화된다. coefficient와 FVE는 Table 2에 제시했다.

Attention Pattern의 mechanistic analysis. 먼저 head j의 position 0 attention score는 input token a의 lookup table(size P)임을 보자. $A_0^j = m x_0^{(0)T} W_K^j W_Q^j x_2^{(0)}$ 이고 token이 항상 '='이므로 $x_2^{(0)}$ 는 constant다. 또한 $x_0^{(0)} = W_E t_0 + p_0$ 이므로 이는 constant vector $C^j = W_E^T W_K^j W_Q^j x_2^{(0)} \in \mathbb{R}^P$ 와 scalar $D = p_0^T W_K^j W_Q^j x_2^{(0)}$ 에 대해 $t_0 \cdot C^j + D$ 로 줄어든다. t_0 는 one-hot이므로 이는 $C^j[a]$ 로 표기할 수 있는 lookup table다.

다음으로 $\Rightarrow 0$ attention pattern은 $\sigma(C^j[a] - C^j[b])$ 다. Appendix A.1처럼 $\Rightarrow =$ attention은 무시하므로 softmax는 두 element의 softmax, 즉 difference의 sigmoid로 줄어든다. C^j 의 꼴은 token index/value에 의존하지 않으므로 position 0과 1에서 같다.

각 head에서 어떤 integer k_j 에 대해 $C_j^i[a] \approx F_j \cos(w_{k_j} a) + G_j \sin(w_{k_j} a)$, 즉 단일 frequency wave로 잘 근사됨을 보인다. C_j^i 를 계산하고 ℓ_2 loss를 최소화하도록 F_j, G_j 를 fit했으며 결과 coefficient는 Figure 10에 표시했다. 이 fit은 각 head variance의 99.02%, 95.21%, 99.10%, 92.42%를 설명한다. 흥미롭게도 head 1과 3의 coefficient는 거의 정확히 서로 반대다.

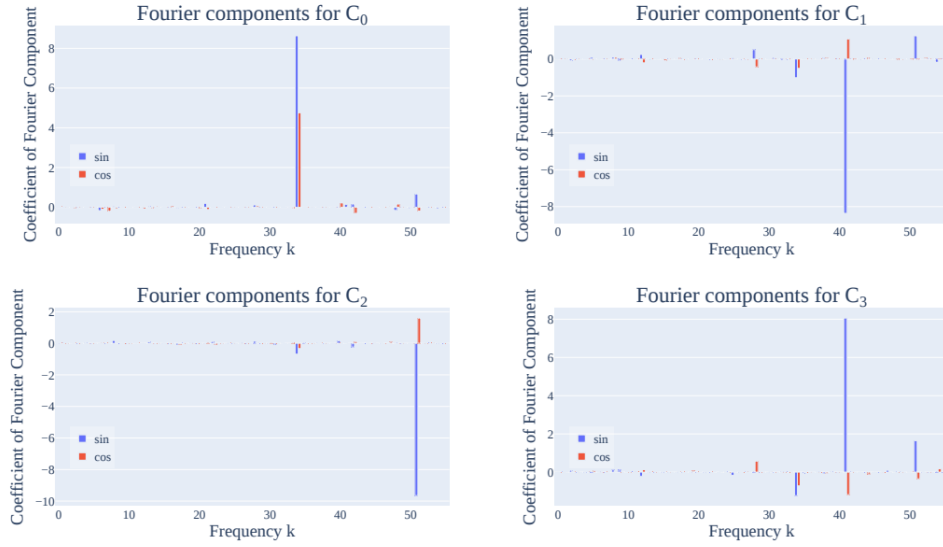


그림 10. 4개 head $j \in \{0, 1, 2, 3\}$ 각각의 attention pattern weight C_j^i 를 Fourier basis에서 표시했다. 각 행 대부분이 단일 frequency와 연관된다는 상당한 sparsity가 관찰된다.

각 head j 에 대해 어떤 constant E_j 가 존재하여 $\sigma(C_j^i[a] - C_j^i[b]) \approx 0.5 + E_j(C_j^i[a] - C_j^i[b])$, 즉 sigmoid는 linear approximation을 갖는다(intercept는 symmetry로 0.5). sigmoid input이 대략 $[-5, 5]$ 의 꽤 넓은 range인데도 이 linear approximation은 variance의 97.5%를 설명한다. sigmoid를 best linear fit으로 바꾸어 확인하면 test loss가 $2.41 \cdot 10^{-7}$ 에서 $2.12 \cdot 10^{-7}$ 로 감소해 성능이 개선된다.

sinusoidal function의 성질에 의해 각 head attention pattern은 대략 $0.5 \pm C_j(\cos(w_{k_j}(a + \theta_j)) - \cos(w_{k_j}(b + \theta_j)))$ 꼴이다. softmax는 intercept 0.5를 갖는 linear map이고 C_j 가 각 token을 단일 frequency score wave로 map하므로, 이는 정확히 Figure 9에 나타난 periodic form을 준다.

마지막으로 각 head j 에 대해 OV circuit output $W_O^j W_V^j x^{(0)}$ 를 Fourier basis에서 plot하고 Figure 11에 보인다. attention pattern weight C_j^i 와 마찬가지로, 유의미한 norm을 갖는 component는 key frequency에 대응하는 것뿐이며 가장 큰 component는 attention pattern C_j^i 의 frequency와 대응한다. head 0과 2는 거의 전적으로 단일 frequency의 sine/cosine으로 구성된다. 반면 head 1과 3은 component norm이 거의 동일하고 5개 key frequency를 모두 포함한다. head 1과 3은 attention pattern weight의 non-constant component가 서로 반대이므로(Table 2, Figure 10), 모든 input에서 attention score 합이 거의 정확히 1이다. 이는 이 두

head가 5개 key frequency에서 first-order term $\sin(w_k)$, $\cos(w_k)$ 를 output하는 데 사용됨을 뜻한다. 우리는 weight decay가 embedding W_E 를 작게 만들도록 유도하여, network가 attention head 두 개를 W_E 크기를 효과적으로 키우는 데 할당하기 때문이라고 추측한다.

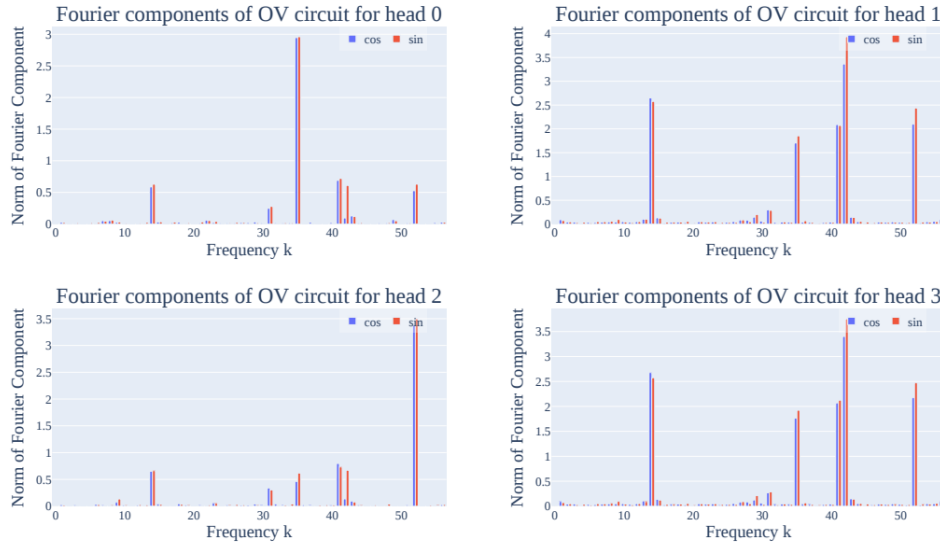


그림 11. 4개 head $j \in \{0, 1, 2, 3\}$ 각각의 OV circuit output $W_O^j W_V^j x^{(0)}$ 를 Fourier basis에서 plot했다. Figure 10의 C^j 와 마찬가지로 유의미한 norm의 component는 key frequency뿐이고, 가장 큰 component는 attention pattern frequency에 대응한다. head 1과 3의 attention pattern 합은 1이며 OV circuit이 거의 같고 5개 key frequency를 모두 포함하므로, 이 두 head는 residual stream의 key frequency magnitude를 키우는 데 사용된다.

정리하면 attention head 0과 2는 각각 단일 frequency cosine/sine의 degree-2 polynomial을 대략 계산하고, head 1과 3은 residual stream에서 key frequency를 증폭한다.

C.1.4. 추가 neuron activation의 periodicity

Figure 12에는 input의 function으로서 MLP neuron 1, 2, 3, 4의 activation을 표시했다. neuron 0과 마찬가지로 이 neuron들의 activation pattern도 두 input 모두에 대해 periodic하다.

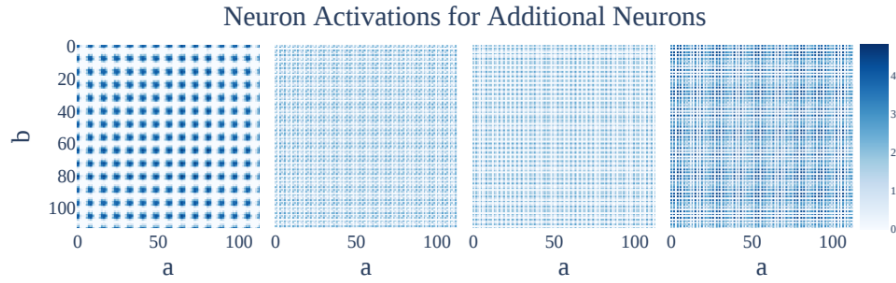


그림 12. input $a, b \in \{0, 1, \dots, 112\}$ 에 대한 MLP neuron 1, 2, 3, 4의 activation plot. neuron 0과 마찬가지로 모든 activation pattern이 두 input에 대해 periodic하다.

C.1.5. mainline run의 추가 Grokking figure

Figure 13은 model을 5개 key frequency만 사용하도록 제한했을 때의 accuracy를 보인다. restricted loss와 마찬가지로, 이는 model이 noise를 제거하기 전부터 modulo generalize하는 방법을 알아낸다는 점을 보여 준다. Figure 14는 logit을 5개 $\cos(w_k(a+b-c))$ term에 regression하여 계산한 logit 내 5개 key frequency coefficient를 보인다. Figure 15는 모든 5개 key frequency를 제외하는 대신, 각각 하나의 key frequency만 제외할 때의 excluded loss를 보인다. 세 figure 모두 Section 5.1에서 논의한 grokking의 관련 phase에 대응하는 inflection point를 보인다.

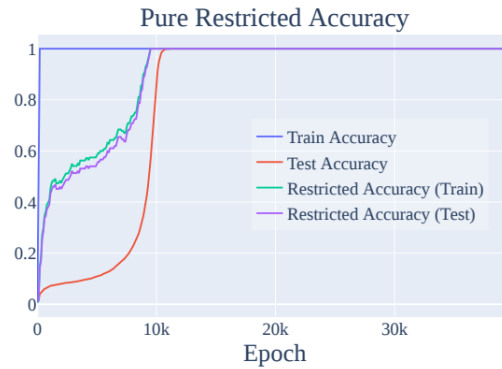


그림 13. Fourier component를 5개 key frequency로 제한했을 때의 accuracy. restricted loss와 마찬가지로, model은 noise를 제거하기 전에 generalize하는 방법을 알아냄을 보여 준다.

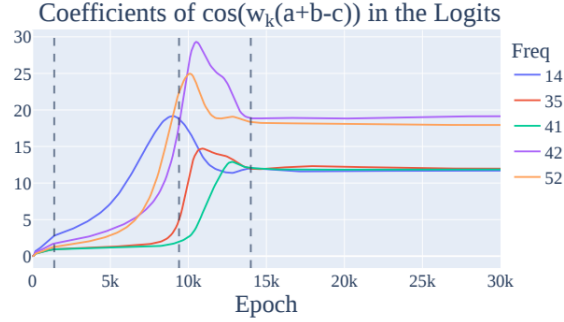


그림 14. model training 동안 logit 내 $\cos(w_k(a+b-c))$ coefficient. 논문의 metric과 마찬가지로 각 cosine term의 매끄러운 interpolation과 성장을 보인다.

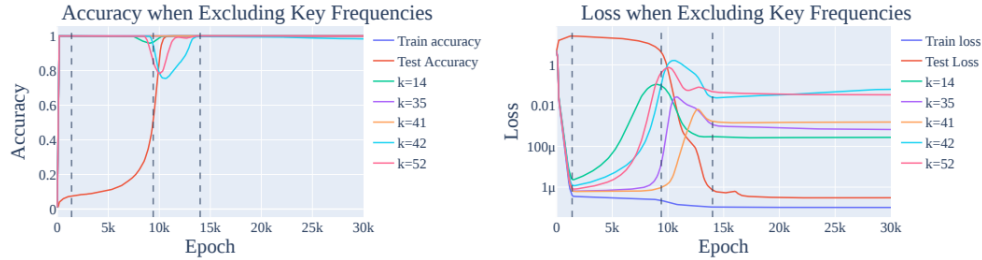


그림 15. mainline model에서 5개 key frequency 각각을 하나씩 제외할 때의 excluded accuracy(왼쪽)와 loss(오른쪽). Section 5.1의 excluded loss 결과와 같이 model이 memorization과 generalization 사이를 interpolate함을 보여 준다.

C.2. 서로 다른 run의 추가 결과

이 절에서는 동일 architecture(Appendix C.2.1) 또는 다른 architecture/experimental setup(Appendix C.2.2)에서의 run으로부터 관련 figure를 제시한다. 일반적으로 모든 model은 modular arithmetic algorithm의 variant를 학습하지만, 사용하는 key frequency 수는 다르다. excluded/restricted loss 계산에 필요한 key frequency를 찾기 위해 neuron-logit map W_L 에 DFT를 수행한 뒤, nontrivial coefficient를 갖는 frequency를 취한다. 이 task의 general(model-independent) progress measure 한 가지는 56개 unique frequency 각각에 대해 excluded loss를 계산해 maximum을 취하는 방식이다. 이 variant의 plot은 대체로 유사하므로 생략한다.

C.2.1. 동일 architecture에서 random seed가 다른 run의 추가 결과

mainline model experimental setup에서 사용한 나머지 4개 run(random seed)이 모두 Fourier multiplication algorithm을 사용한다는 evidence를 제시하고, 이 run에서도 같은 grokking phase가 발생함을 확인한다.

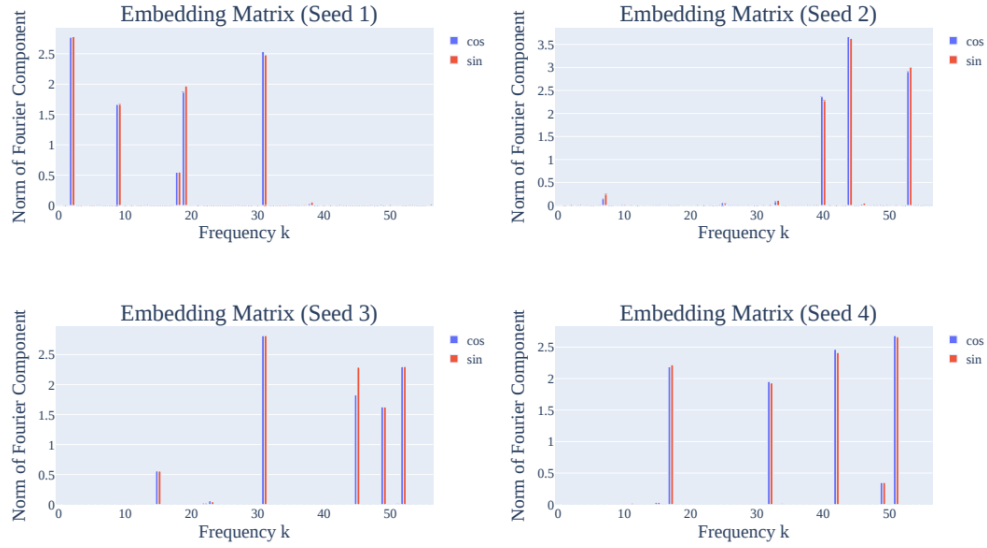


그림 16. 원래 one-layer architecture의 다른 4개 random seed에서 embedding matrix W_E Fourier component norm. Section 4.1과 Appendix C.2.1에서 논의하듯 Fourier basis에서 W_E 가 sparse하다는 것은 network가 Fourier basis에서 작동한다는 증거다.

Figure 16은 다른 4개 seed의 W_E Fourier component norm을 보인다. mainline model처럼 matrix는 Fourier basis에서 sparse하다. Figure 17은 다른 4개 seed의 neuron-logit map W_L Fourier component norm을 보인다. 이 matrix도 Fourier basis에서 sparse해서, seed별로 3 또는 4개의 key frequency를 식별할 수 있다. 단, 구체적 frequency는 seed마다 다르다.

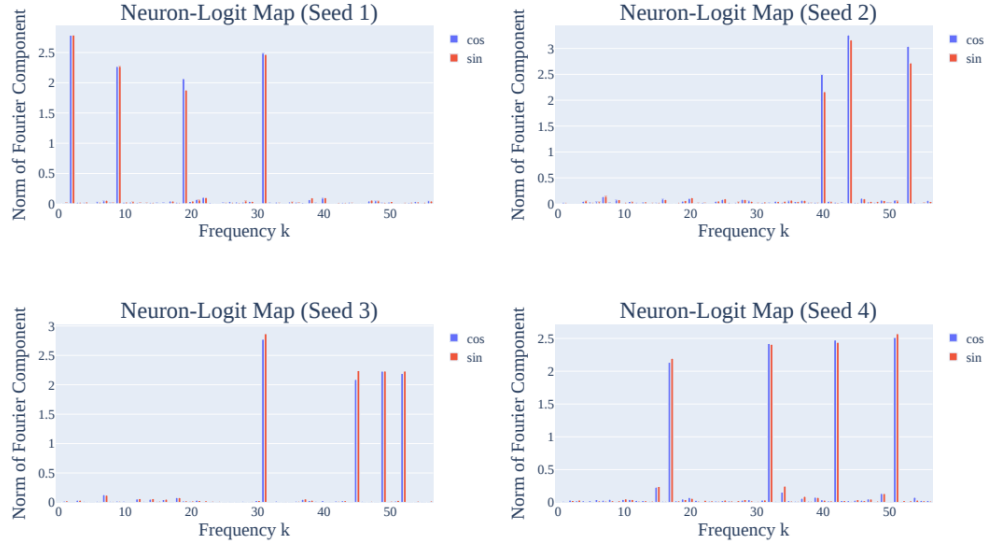


그림 17. neuron-logit map weight W_L 에서 sine/cosine wave에 대응하는 direction norm. 본문의 mainline model 및 Appendix C.2.1과 같이 W_L 은 일관되게 sparse하며, 4개 model 모두 Fourier basis에서 작동한다는 증거를 제공한다.

neuron-logit map에서 식별한 key frequency로 Section 4.2 experiment를 반복하여, key frequency에 대응하는 6 또는 8 direction의 MLP activation을 “read off”한다. mainline model처럼 MLP layer가 $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ trigonometric identity를 계산함을 확인한다. activation을 해당 단일 term으로 근사하여 확인한 결과 FVE는 mainline model처럼 일관되게 거의 100%다. 다음으로 Section 4.4와 같이 logit의 key frequency를 ablate한다(Table 4). mainline model과 동일하게 모든 key frequency ablation은 chance보다도 나쁜 성능을 만들지만, key frequency만 남기고 나머지를 모두 ablate하면 test performance가 좋아진다.

W_L Component	Fourier components of $u_k^T \text{MLP}(a, b)$ or $v_k^T \text{MLP}(a, b)$	FVE
$\cos(w_2c)$	$147.4 \cos(w_2a) \cos(w_2b) - 145.8 \sin(w_2a) \sin(w_2b) \approx 146.6 \cos(w_2(a+b))$	99.2%
$\sin(w_2c)$	$145.5 \cos(w_2a) \sin(w_2b) + 145.6 \sin(w_2a) \cos(w_2b) \approx 145.5 \sin(w_2(a+b))$	99.1%
$\cos(w_9c)$	$49.3 \cos(w_9a) \cos(w_9b) - 48.0 \sin(w_9a) \sin(w_9b) \approx 48.6 \cos(w_9(a+b))$	96.4%
$\sin(w_9c)$	$48.6 \cos(w_9a) \sin(w_9b) + 48.5 \sin(w_9a) \cos(w_9b) \approx 48.5 \sin(w_9(a+b))$	96.7%
$\cos(w_{19}c)$	$58.0 \cos(w_{19}a) \cos(w_{19}b) - 58.3 \sin(w_{19}a) \sin(w_{19}b) \approx 58.2 \cos(w_{19}(a+b))$	95.4%
$\sin(w_{19}c)$	$59.3 \cos(w_{19}a) \sin(w_{19}b) + 59.4 \sin(w_{19}a) \cos(w_{19}b) \approx 59.4 \sin(w_{19}(a+b))$	93.9%
$\cos(w_{31}c)$	$94.4 \cos(w_{31}a) \cos(w_{31}b) - 96.4 \sin(w_{31}a) \sin(w_{31}b) \approx 95.4 \cos(w_{31}(a+b))$	98.4%
$\sin(w_{31}c)$	$97.2 \cos(w_{31}a) \sin(w_{31}b) + 97.1 \sin(w_{31}a) \cos(w_{31}b) \approx 97.2 \sin(w_{31}(a+b))$	98.7%
(a) Seed 1		
W_L Component	Fourier components of $u_k^T \text{MLP}(a, b)$ or $v_k^T \text{MLP}(a, b)$	FVE
$\cos(w_{40}c)$	$97.0 \cos(w_{40}a) \cos(w_{40}b) - 99.4 \sin(w_{40}a) \sin(w_{40}b) \approx 98.2 \cos(w_{40}(a+b))$	97.3%
$\sin(w_{40}c)$	$81.3 \cos(w_{40}a) \sin(w_{40}b) + 81.3 \sin(w_{40}a) \cos(w_{40}b) \approx 81.3 \sin(w_{40}(a+b))$	92.7%
$\cos(w_{44}c)$	$309.1 \cos(w_{44}a) \cos(w_{44}b) - 338.7 \sin(w_{44}a) \sin(w_{44}b) \approx 323.9 \cos(w_{44}(a+b))$	98.5%
$\sin(w_{44}c)$	$327.3 \cos(w_{44}a) \sin(w_{44}b) + 327.2 \sin(w_{44}a) \cos(w_{44}b) \approx 327.3 \sin(w_{44}(a+b))$	98.9%
$\cos(w_{53}c)$	$192.1 \cos(w_{53}a) \cos(w_{53}b) - 192.2 \sin(w_{53}a) \sin(w_{53}b) \approx 192.1 \cos(w_{53}(a+b))$	97.3%
$\sin(w_{53}c)$	$166.7 \cos(w_{53}a) \sin(w_{53}b) + 166.8 \sin(w_{53}a) \cos(w_{53}b) \approx 166.8 \sin(w_{53}(a+b))$	95.7%
(b) Seed 2		
W_L Component	Fourier components of $u_k^T \text{MLP}(a, b)$ or $v_k^T \text{MLP}(a, b)$	FVE
$\cos(w_{31}c)$	$156.1 \cos(w_{31}a) \cos(w_{31}b) - 156.5 \sin(w_{31}a) \sin(w_{31}b) \approx 156.3 \cos(w_{31}(a+b))$	99.3%
$\sin(w_{31}c)$	$150.7 \cos(w_{31}a) \sin(w_{31}b) + 150.7 \sin(w_{31}a) \cos(w_{31}b) \approx 150.7 \sin(w_{31}(a+b))$	98.9%
$\cos(w_{45}c)$	$72.5 \cos(w_{45}a) \cos(w_{45}b) - 76.8 \sin(w_{45}a) \sin(w_{45}b) \approx 74.6 \cos(w_{45}(a+b))$	95.9%
$\sin(w_{45}c)$	$74.7 \cos(w_{45}a) \sin(w_{45}b) + 74.6 \sin(w_{45}a) \cos(w_{45}b) \approx 74.6 \sin(w_{45}(a+b))$	96.6%
$\cos(w_{49}c)$	$45.9 \cos(w_{49}a) \cos(w_{49}b) - 45.5 \sin(w_{49}a) \sin(w_{49}b) \approx 45.7 \cos(w_{49}(a+b))$	97.0%
$\sin(w_{49}c)$	$45.8 \cos(w_{49}a) \sin(w_{49}b) + 45.8 \sin(w_{49}a) \cos(w_{49}b) \approx 45.8 \sin(w_{49}(a+b))$	96.9%
$\cos(w_{52}c)$	$71.6 \cos(w_{52}a) \cos(w_{52}b) - 72.1 \sin(w_{52}a) \sin(w_{52}b) \approx 71.9 \cos(w_{52}(a+b))$	98.5%
$\sin(w_{52}c)$	$68.7 \cos(w_{52}a) \sin(w_{52}b) + 68.7 \sin(w_{52}a) \cos(w_{52}b) \approx 68.7 \sin(w_{52}(a+b))$	97.9%
(c) Seed 3		
W_L Component	Fourier components of $u_k^T \text{MLP}(a, b)$ or $v_k^T \text{MLP}(a, b)$	FVE
$\cos(w_{17}c)$	$66.0 \cos(w_{17}a) \cos(w_{17}b) - 63.5 \sin(w_{17}a) \sin(w_{17}b) \approx 64.8 \cos(w_{17}(a+b))$	96.4%
$\sin(w_{17}c)$	$66.4 \cos(w_{17}a) \sin(w_{17}b) + 66.4 \sin(w_{17}a) \cos(w_{17}b) \approx 66.4 \sin(w_{17}(a+b))$	94.9%
$\cos(w_{32}c)$	$68.7 \cos(w_{32}a) \cos(w_{32}b) - 68.4 \sin(w_{32}a) \sin(w_{32}b) \approx 68.5 \cos(w_{32}(a+b))$	96.2%
$\sin(w_{32}c)$	$68.0 \cos(w_{32}a) \sin(w_{32}b) + 68.0 \sin(w_{32}a) \cos(w_{32}b) \approx 68.0 \sin(w_{32}(a+b))$	96.3%
$\cos(w_{42}c)$	$100.4 \cos(w_{42}a) \cos(w_{42}b) - 96.0 \sin(w_{42}a) \sin(w_{42}b) \approx 98.2 \cos(w_{42}(a+b))$	97.9%
$\sin(w_{42}c)$	$100.2 \cos(w_{42}a) \sin(w_{42}b) + 100.1 \sin(w_{42}a) \cos(w_{42}b) \approx 100.1 \sin(w_{42}(a+b))$	98.6%
$\cos(w_{51}c)$	$118.0 \cos(w_{51}a) \cos(w_{51}b) - 116.2 \sin(w_{51}a) \sin(w_{51}b) \approx 117.1 \cos(w_{51}(a+b))$	99.0%
$\sin(w_{51}c)$	$114.3 \cos(w_{51}a) \sin(w_{51}b) + 114.2 \sin(w_{51}a) \cos(w_{51}b) \approx 114.2 \sin(w_{51}(a+b))$	98.5%
(d) Seed 4		

표 3. 다른 4개 random seed의 final model neuron-logit map W_L direction마다 해당 direction으로 MLP activation을 project하고 Fourier transform를 수행했다. 간결성을 위해 가장 큰 coefficient의 15% 미만 항은 생략했다. projection을 $\cos(w_k(a+b))$ 또는 $\sin(w_k(a+b))$ 단일 항의 배수로 대체했을 때 FVE가 일관되게 1에 가까움을 보인다.

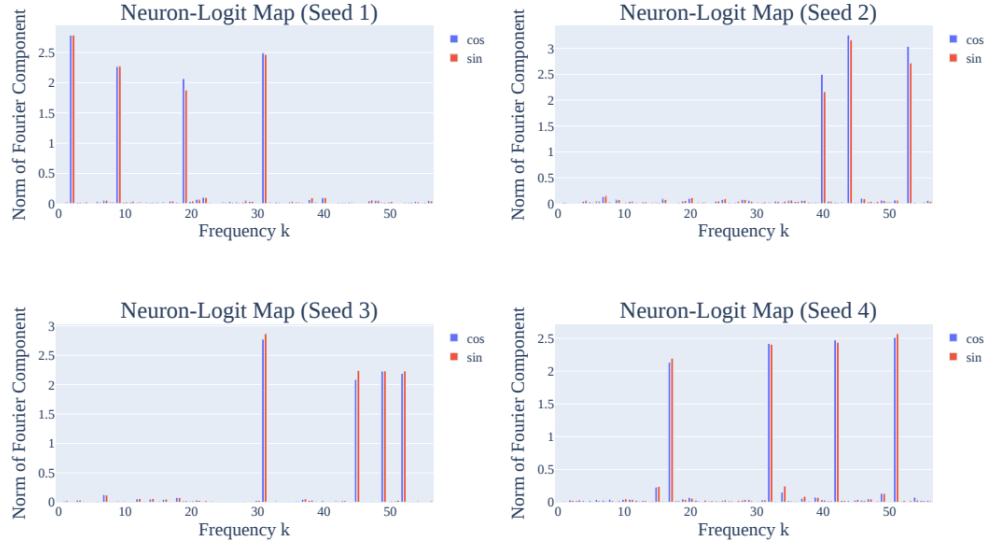


Figure 17: The norms of the direction corresponding to sine and cosine waves in the neuron-logit map weights W_L . As with the mainline model discussed in the main body and discussed in Appendix C.2.1, W_L is consistently sparse, providing evidence that all four are operating in a Fourier basis.

Seed	Test Loss	Loss (Key frequencies removed)	Loss (All other frequencies removed)
1	$2.07 \cdot 10^{-7}$	$6.5 \cdot 10^0$	$5.7 \cdot 10^{-8}$
2	$2.1 \cdot 10^{-7}$	$1.1 \cdot 10^1$	$6.2 \cdot 10^{-8}$
3	$2.05 \cdot 10^{-7}$	$6.7 \cdot 10^0$	$5.5 \cdot 10^{-8}$
4	$2.33 \cdot 10^{-7}$	$6.8 \cdot 10^0$	$6.0 \cdot 10^{-8}$

표 4. 각 network에서 key frequency를 ablate하면 성능이 chance보다도 나빠지는 반면, 나머지 모든 frequency를 ablate하면 성능이 개선됨을 확인한다.

Progress measure와 grokking. 마지막으로 동일 architecture의 다른 run에서 mainline model progress measure 및 grokking 결과를 확인한다. Figure 18은 다른 4개 seed의 train/test/restricted loss를, Figure 19는 embedding matrix W_E 와 neuron-logit map W_L Fourier component의 Gini coefficient를 보인다. curve shape는 mainline model과 매우 유사하여, 이 model의 grokking도 본문의 세 phase로 나눌 수 있다. 흥미롭게도 모든 model이 약 1,400 epoch에 memorization을 끝내지만 circuit formation과 cleanup 시점은 서로 다르다.

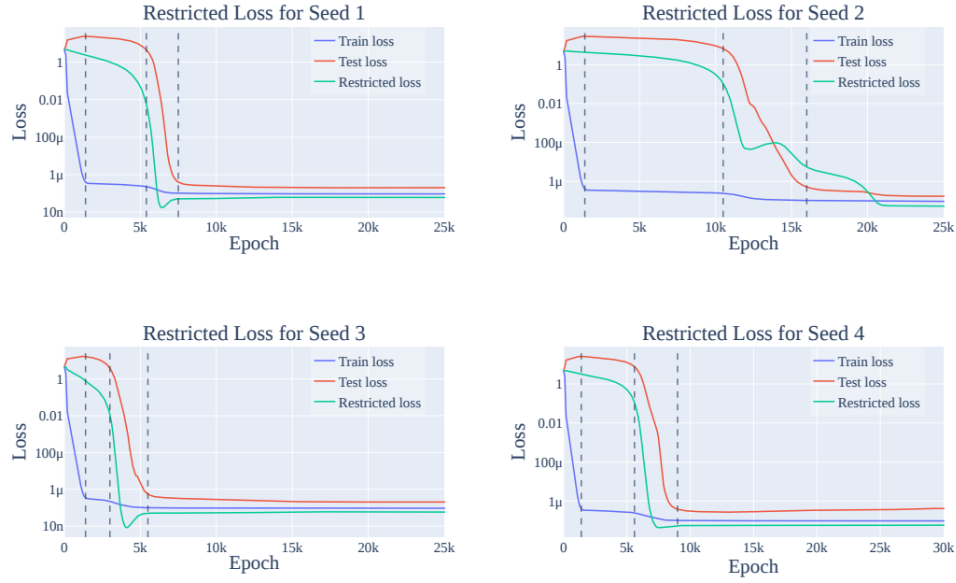


그림 18. Appendix C.2.1의 다른 4개 random seed에서 train, test, restricted loss. 선은 memorization, circuit formation, cleanup(마지막 stable phase 포함)을 구분한다. mainline model처럼 restricted loss가 train loss보다 앞서 일관되게 감소한다. curve shape는 서로 및 mainline model과 유사하지만 grokking 시점과 phase divider는 seed마다 다르다. 모든 5개 run에서 memorization은 약 1,400 step에 완료된다.

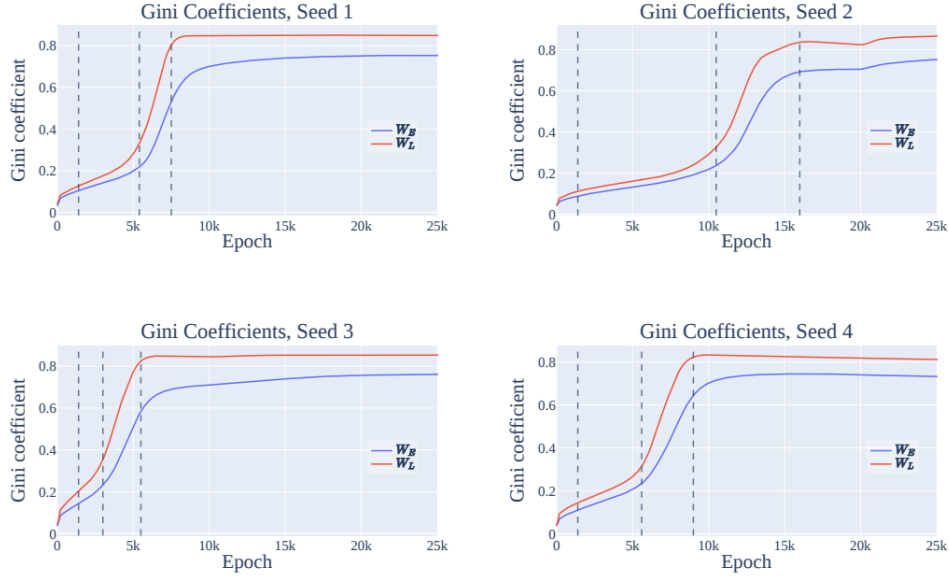


그림 19. 다른 4개 random seed에서 embedding matrix W_E 와 neuron-logit map W_L Fourier component의 Gini coefficient(sparsity measure). 선은 training의 세 phase를 구분한다. mainline model처럼 sparsity는 memorization/circuit formation 중 천천히 증가하고 cleanup 중 빠르게 증가한다.

C.2.2. 다른 experimental setup 결과

이 절에서는 network size, training data amount, prime P size를 변화시켜 small transformer가 modular addition task에서 grok한다는 추가 evidence를 제시한다.

Training data fraction을 변화시킨 1-layer Transformer. Figure 20처럼 $P=113$ modular addition task에서 여러 data fraction, 즉 training 중 model이 보는 $113 \cdot 113$ input pair 비율에서 grokking이 발생한다. data 양이 작을수록 grokking은 느리고, data fraction이 충분히 크면($\geq 60\%$) Figure 20과 21처럼 즉시 generalization이 일어난다. Appendix C.2.1의 결과처럼 이 절의 모든 1-layer transformer도 Fourier multiplication algorithm을 사용하는 쪽으로 수렴한다.

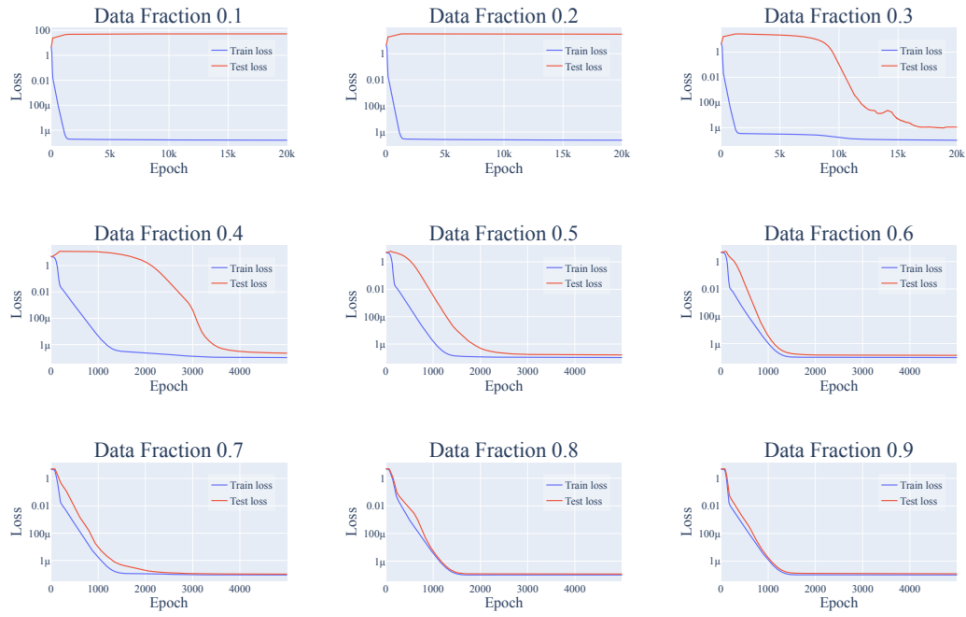


그림 20. training에 사용하는 113·113 possible input pair 비율을 바꾼 P=113 modular addition task one-layer transformer의 training/test loss. dataset의 30~50%를 쓸 때 grokking이 발생하고, 더 적은 data fraction은 grokking을 늦춘다. data $\geq 60\%$ 에서는 즉시 generalization이 일어나고, 10% 또는 20%에서는 40k epoch 뒤에도 grokking이 발생하지 않는다. detail을 위해 data fraction $\geq 40\%$ run에는 x-axis를 5k epoch까지만 표시했다.

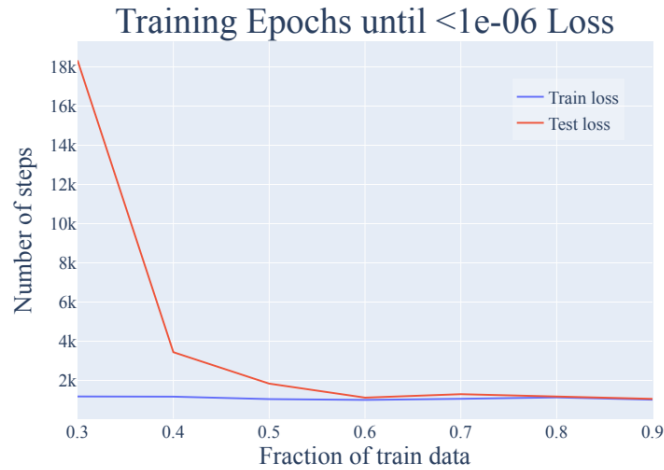


그림 21. training data 양의 function으로서 train/test loss가 $<10^{-6}$ 가 되는 step 수. train loss는 모든 data fraction에서 즉시 10^{-6} 아래로 수렴하지만, data fraction이 작을수록 generalization에는 훨씬 더 오래 걸린다. 다른 threshold의 plot도 정성적으로 유사하다.

2-layer Transformer. Figure 22처럼 2-layer transformer도 어느 정도 grokking을 보인다. 그러나 이는 slingshot mechanism(Thilak et al., 2022) 때문에 복잡해진다. Figure 23은 2-layer transformer의 excluded loss를 보이며, grokking 이전에 relatively smooth하게 개선된다는 점에서 mainline one-layer transformer와 비슷한 pattern을 보인다.

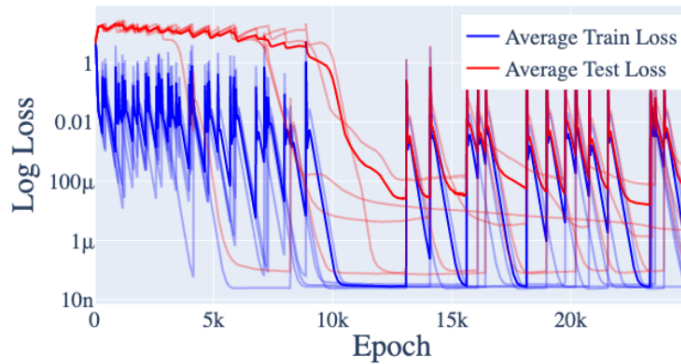


그림 22. original architecture의 2-layer version에서 training/test loss. 5개 random seed 평균은 굵게 표시했다.

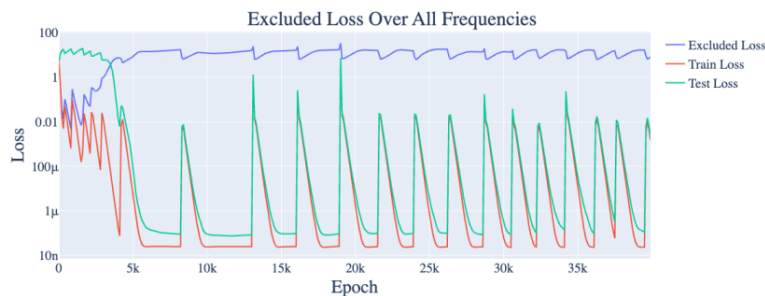


그림 23. original architecture의 2-layer version에서 training, test, full excluded loss. 가독성을 위해 random seed 하나를 골랐다.

더 작고 큰 prime. 더 작거나 큰 prime modulus도 조사했다. $P=53$ (Figure 24)에서는 작은 prime case에서 grokking을 관찰하기 위해 여러 weight decay를 탐색했다. original $\lambda=1$ setting에서는 model이 generalize하지 않았지만, weight decay를 $\lambda=5$ 로 늘리면 grokking할 수 있다. 이는 total pair가 $53 \cdot 53$ 뿐이라 memorization solution이 훨씬 작고, generalizing solution이 선호되려면 더 aggressive한 weight decay가 필요하기 때문이라고 추측한다. $P=109$ 에서는 mainline model과 정확히 같은 behavior를 보았다.

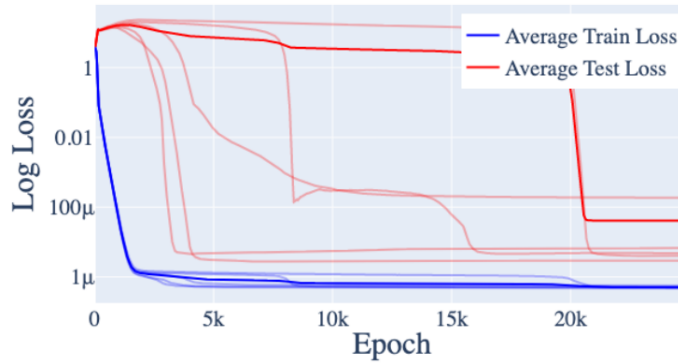


그림 24. P=53에서 weight decay($\lambda=5$)를 제외한 hyperparameter는 main training run과 동일할 때의 training/test loss. average는 굵게, 개별 run은 부분 투명도로 표시했다. grokking이 발생한다.

P=401(Figure 25)에서는 $\lambda \in \{0.3, 0.5, 1, 3, 5, 8\}$ 로 weight decay를 바꾸어도 grokking을 얻을 수 없었다. 대신 model은 즉시 generalizing solution을 학습했다. 이는 P=113일 때 30%의 113·113 pair를 본 것에 비해, P=401에서 30%의 401·401 pair를 보므로 model이 경험하는 data 양이 크게 늘어 generalizing solution이 처음부터 선호되기 때문이라고 믿는다. 이어 $\lambda=1$ 에서 5%, 10%, 20% training pair를 사용해 각각 model 3개를 학습했다. 5%와 10% model은 즉시 overfit하고 generalize하지 않았고, 20% model은 역시 즉시 generalize했다.

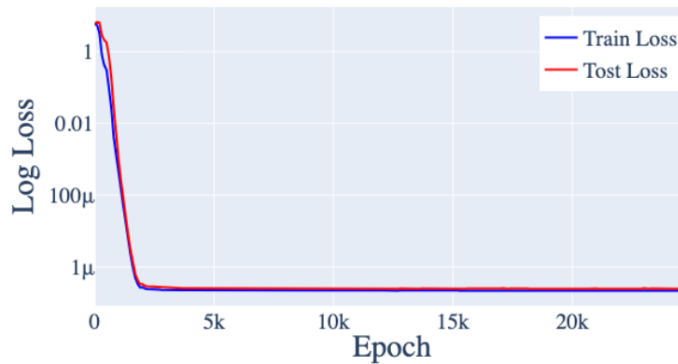


그림 25. P=401에서 main training run과 다른 hyperparameter는 동일한 경우의 training/test loss. 여러 weight decay에 걸쳐 grokking은 일어나지 않고 model은 즉시 generalize한다.

C.2.3. Generalizing model은 일관되게 Fourier multiplication algorithm을 사용한다

Appendix C.2.2에서 low test loss를 얻은 각 model에 대해 mainline model에 수행한 분석을 반복하고 결과를 Table 5에 요약했다. key frequency, Gini coefficient, 관련 FVE를 나열한다. weight decay로 학습되어 정확히 generalize하는 모든 model이 Fourier multiplication algorithm의 어떤 variation을 구현함을 발견했다.

흥미롭게도 dropout으로 학습한 model의 embedding/unembedding matrix는 Fourier basis에서 sparse하지 않고, $p=0.2$ model logit은 다른 model만큼 cosine sum으로 잘 설명되지 않는다. 이는 dropout model이 grok하는 데 더 오래 걸리는 듯해 training epoch가 부족했던 점과, dropout training network에 redundancy가 본질적으로 필요하다는 점이 결합한 결과일 가능성이 있다고 추측한다. mainline model처럼 generalizing model은 final MLP 주위 final skip connection을 무시한다. 연구한 generalizing model은 skip connection을 zero/mean ablate해도 유의미한 performance penalty를 겪지 않는다(Table 6).

Model	Test Loss	Gini(W_E)	Gini(W_L)	Key Frequencies	Logit FVE	MLP FVE
40% Training Data	$1.98 \cdot 10^{-7}$	0.76	0.79	[17, 43, 49, 55]	94.9%	83.3% [26.1%]
50% Training Data	$1.68 \cdot 10^{-7}$	0.75	0.77	[2, 17, 31, 41, 44]	91.2%	85.2% [28.2%]
60% Training Data	$1.23 \cdot 10^{-7}$	0.79	0.84	[2, 23, 34, 51]	96.4%	95.7% [1.4%]
70% Training Data	$9.85 \cdot 10^{-8}$	0.80	0.91	[14, 15, 26]	99.0%	98.9% [0.4%]
80% Training Data	$5.83 \cdot 10^{-7}$	0.62	0.80	[38, 41]	63.9%	94.1% [2.5%]
90% Training Data	$1.11 \cdot 10^{-7}$	0.79	0.88	[3, 26, 34, 43]	98.6%	98.7% [0.3%]
2 Layer Transformer	$9.54 \cdot 10^{-7}$	0.59	0.80	[14, 18, 29]	91.8%	95.2% [1.9%]
2 Layer Transformer	$4.41 \cdot 10^{-5}$	0.55	0.73	[7, 12, 35, 49]	86.1%	86.2% [6.4%]
2 Layer Transformer	$6.50 \cdot 10^{-2}$	0.66	0.80	[4, 9, 28]	88.5%	85.4% [5.9%]
2 Layer Transformer	$4.18 \cdot 10^{-2}$	0.56	0.76	[4, 5, 15, 54]	91.4%	81.2% [17.8%]
2 Layer Transformer	$1.75 \cdot 10^{-2}$	0.68	0.71	[3, 4, 13, 30, 38]	84.0%	71.9% [19.5%]
$P = 53$	$3.00 \cdot 10^{-4}$	0.61	0.68	[6, 9, 16, 21]	91.2%	90.2% [5.8%]
$P = 53$	$1.03 \cdot 10^{-4}$	0.56	0.72	[4, 13, 16]	94.8%	93.1% [6.4%]
$P = 53$	$1.21 \cdot 10^{-5}$	0.66	0.79	[13, 22, 23]	98.2%	97.6% [0.9%]
$P = 53$	$3.95 \cdot 10^{-6}$	0.66	0.74	[3, 14, 15]	88.5%	91.8% [4.6%]
$P = 53$	$5.56 \cdot 10^{-6}$	0.67	0.80	[10, 14, 22]	98.1%	98.3% [0.6%]
$P = 109$	$2.02 \cdot 10^{-7}$	0.76	0.83	[6, 7, 22, 25]	98.0%	97.3% [1.9%]
$P = 109$	$2.95 \cdot 10^{-7}$	0.69	0.82	[8, 14, 29, 32, 41]	95.2%	94.7% [2.3%]
$P = 109$	$1.66 \cdot 10^{-7}$	0.78	0.86	[13, 23, 39, 45]	98.5%	97.6% [0.9%]
$P = 109$	$2.50 \cdot 10^{-7}$	0.68	0.82	[8, 13, 32, 41]	96.8%	95.5% [2.3%]
$P = 109$	$2.77 \cdot 10^{-7}$	0.76	0.85	[29, 37, 38, 49]	97.9%	98.1% [0.8%]
Dropout $p = 0.2$	$2.65 \cdot 10^{-1}$	0.19	0.46	[1, 4, 7, 17, 22, 33, 40, 49, 55]	71.3%	65.0% [17.5%]
Dropout $p = 0.2$	$4.52 \cdot 10^{-1}$	0.19	0.46	[3, 8, 19, 28, 32, 34, 40, 44]	73.3%	71.4% [10.7%]
Dropout $p = 0.2$	$2.03 \cdot 10^{-1}$	0.20	0.45	[4, 5, 32, 38, 41, 44, 49, 50]	74.2%	71.1% [10.6%]
Dropout $p = 0.5$	$< 10^{-8}$	0.26	0.56	[1, 4, 26, 46, 47, 55]	89.4%	88.9% [3.5%]
Dropout $p = 0.5$	$2.01 \cdot 10^{-2}$	0.20	0.49	[16, 21, 35, 47, 53]	88.4%	88.4% [3.0%]
Dropout $p = 0.5$	$< 10^{-8}$	0.25	0.54	[1, 4, 7, 19, 29, 31, 42]	86.1%	85.6% [4.0%]

표 5. Appendix C.2.3 및 D.1의 test data generalizing model에 대해 test loss, W_E/W_L Fourier component norm Gini coefficient(Section 5.1), key frequency, 그리고 key frequency에서의 $\cos(w_k(a+b-c))$ weighted sum이 설명하는 logit variance 비율(Section 4.2)을 보고한다. 또한 key frequency cosine/sine에 대응하는 W_L component u_k, v_k 를 찾고, 단일 $\cos(w_k(a+b))$ 또는 $\sin(w_k(a+b))$ term이 설명하는 $u_k^T \text{MLP}(a,b), v_k^T \text{MLP}(a,b)$ variance의 평균 비율을 보고한다. 대괄호 수치는 standard deviation이며 2-layer model은 final layer MLP activation을 쓴다. 이 목록에서 dropout $p=0.2$ model을 제외한 모든 model은 test accuracy $>99.95\%$, $p=0.2$ model은 약 99.6% 다.

In addition, we find the components u_k, v_k of W_L that correspond to cosines and sines of the key frequencies, and then report the average fraction of variance of $u_k^T \text{MLP}(a, b)$ and $v_k^T \text{MLP}(a, b)$ explained by a single term of form $\cos(w_k(a + b))$ or $\sin(w_k(a + b))$ respectively (Section 4.2). Numbers in square brackets represent the standard deviation. For 2 Layer models, we use the final layer MLP activations for $\text{MLP}(a, b)$.

We omit test accuracy because every model on this list except for the dropout $p = 0.2$ models achieves $> 99.95\%$ test accuracy, while the dropout $p = 0.2$ models achieve around 99.6% test accuracy.

Model Type	Loss	Accuracy	Ablated Loss	Ablated Accuracy
Varying Data Fraction	$1.83 \cdot 10^{-7}$ ($1.65 \cdot 10^{-7}$)	100%	$7.74 \cdot 10^{-7}$ ($6.74 \cdot 10^{-7}$)	100%
2 Layer Transformer	$1.97 \cdot 10^{-2}$ ($2.41 \cdot 10^{-2}$)	99.6%	$4.63 \cdot 10^{-2}$ ($6.72 \cdot 10^{-2}$)	98.7%
$P = 53$	$5.96 \cdot 10^{-5}$ ($8.91 \cdot 10^{-5}$)	100%	$1.5 \cdot 10^{-4}$ ($2.70 \cdot 10^{-4}$)	100%
$P = 109$	$1.94 \cdot 10^{-7}$ ($3.74 \cdot 10^{-8}$)	100%	$6.53 \cdot 10^{-7}$ ($1.41 \cdot 10^{-7}$)	100%
Dropout $p = 0.2$	0.215 (0.091)	99.7%	0.205 (0.075)	99.7%
Dropout $p = 0.5$	$4.68 \cdot 10^{-3}$ ($8.11 \cdot 10^{-3}$)	100%	$3.6 \cdot 10^{-3}$ ($5.82 \cdot 10^{-3}$)	100%

표 6. final MLP layer 주위 skip connection의 중요성을 확인하기 위해 skip connection을 mean ablate하고, 각 problem에서 전체 dataset의 loss/accuracy를 모든 run 평균으로 계산했다(loss standard deviation은 괄호에 보고). loss는 조금 증가하지만 accuracy는 계속 높고 ablated model loss도 낮다. zero ablation 결과도 유사하다.

APPENDIX D. GROKking의 추가 결과

D.1. Grokking에는 regularization과 limited data가 모두 필요하다

Section 7과 Appendix C.2에서 논의했듯, weight decay와 data 양은 grokking 발생 여부에 큰 영향을 준다. 이를 확인하기 위해 one-layer transformer에서 weight decay를 제거하고 data amount를 바꿔 experiment했다. Figure 26은 $\lambda=0$ (weight decay 없음)인 typical training run의 train/test/full excluded loss다. figure처럼 grokking은 발생하지 않고 excluded loss도 증가하지 않는다. 이는 model이 generalizing algorithm circuit을 전혀 형성하지 않음을 시사한다.

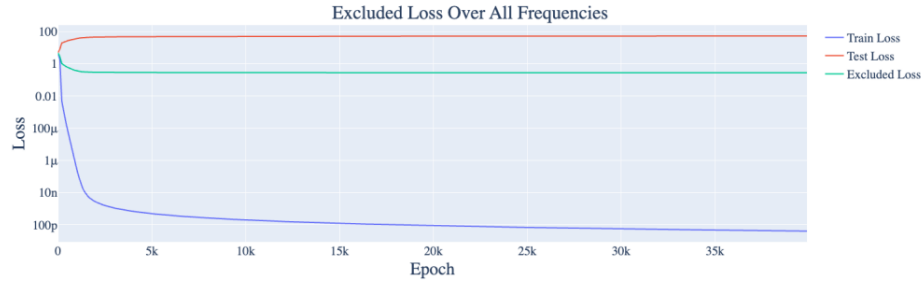


그림 26. weight decay 없이 학습한 original architecture one-layer version의 training, test, full excluded loss. 가독성을 위해 random seed 하나를 골랐다. weight decay가 없으면 grokking이 방지된다.

Figure 20은 $\lambda=1$ weight decay와 여러 data fraction에서 학습한 model의 test loss curve를 보인다. 모든 train loss는 대략 같아 같은 rate로 memorize하지만, data fraction이 더 작은 model일수록 grokking하는 데 더 오래 걸린다. Figure 27은 $\lambda=0.3$ 과 $\lambda=3.0$ 으로 학습한 model의 test/train loss를 보인다. 더 작은 weight decay는 grokking을 느리게 하고 더 큰 weight decay는 더 빠르게 한다. 평균적으로 $\lambda=0.3$ 에서는 약 3k epoch, $\lambda=1.0$ 에서는 5~10k epoch, $\lambda=3.0$ 에서는 20k epoch가 걸린다.

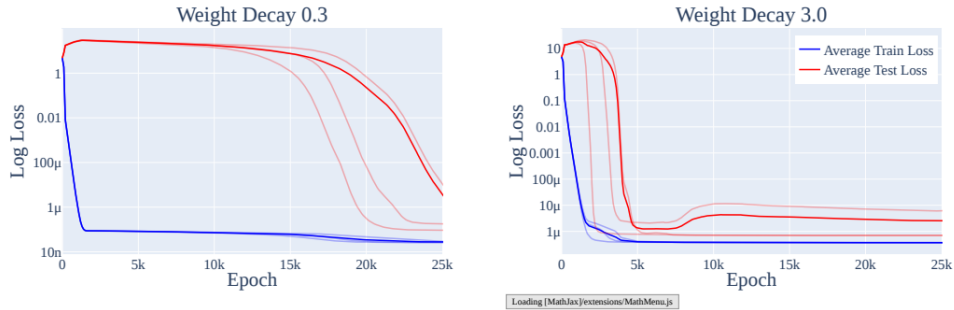


그림 27. $\lambda=0.3$ (왼쪽)과 $\lambda=3.0$ (오른쪽) weight decay에서 training 중 train/test loss. 덜 aggressive한 weight decay는 더 느린 grokking으로 이어진다.

마지막으로 다른 regularization도 grokking을 유도하는지 검증한다. weight decay를 다음 regularization으로 바꾸고 다른 hyperparameter는 유지했다.

1. **Dropout.** MLP neuron에 Srivastava et al. (2014)의 dropout을 추가하고 $p \in \{0.2, 0.5, 0.8\}$ 을 사용했다. 각 neuron을 training 중 probability p 로 0으로 두고, 나머지 neuron output에는 $1/(1-p)$ 를 곱한다.
2. **ℓ_1 Regularization.** loss term에 ℓ_1 penalty를 추가하고 $\lambda \in \{1, 10, 100\}$ 을 사용했다. ℓ_2 regularization을 AdamW(Loshchilov & Hutter, 2017)로 수행할 때처럼, ℓ_1 penalty에 대한 update를 log loss optimization step에서 decouple하지는 않는다.

각 case에서 random seed 세 개를 수행했다. Figure 28은 결과를 보인다. ℓ_1 regularization에서는 grokking이 발생하지 않았으나, $p=0.2$ 또는 $p=0.5$ dropout에서는 세 seed 모두 grokking이 발생했다. dropout과 weight decay는 Fourier multiplication algorithm에 필요한 computation spread-out을 장려하는 반면, ℓ_1 regularization은 network가 neuron basis에서 더 sparse하고 Fourier basis에서는 덜 sparse해지도록 하므로 Fourier multiplication algorithm 학습을 막는다고 추측한다.

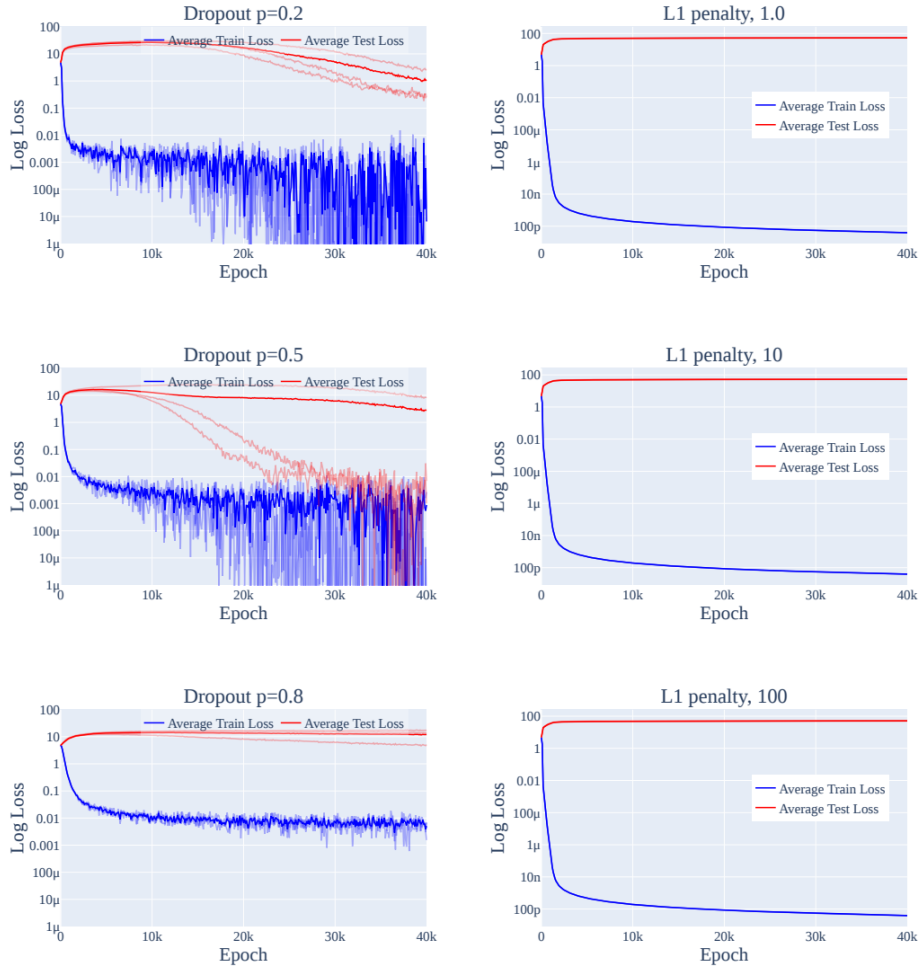


그림 28. 두 종류 regularization, dropout과 ℓ_1 regularization에서 training 중 train/test loss. dropout에서는 일부 run에서 grokking이 발생하지만 ℓ_1 regularization에서는 발생하지 않는다.

D.2. Slingshot mechanism은 자주 발생하지만 Grokking에 필요하지는 않다

Section C.2에서 언급했듯, 2-layer transformer는 training 중 상당한 slingshot(Thilak et al., 2022)을 보인다. 이는 서로 다른 scale gradient가 adaptive optimizer에서 상호작용하기 때문이라고 추측한다. loss calculation precision을 낮추어 많은 gradient가 0으로 round되도록 해서, gradient scale 차이를 크게 만들면 one-layer model에서도 slingshot을 유도할 수 있었다.

그러나 많은 one-layer model은 slingshot 없이도 grok하므로, weight decay나 다른 regularization이 있을 때 grokking에 slingshot mechanism은 필요하지 않다. weight decay 없는 training run에서 grokking과 동반되는 Thilak et al. (2022)의 slingshot은, 더 복잡한 memorizing solution보다 더 단순한 generalizing solution을 선호하도록 만드는 implicit regularization mechanism으로 작동할 수 있다고 추측한다.

D.3. 다른 algorithmic task의 추가 evidence

추가 algorithmic task 세 가지에서 grokking phenomenon을 분석하고 limited data가 grokking의 중요한 부분임을 확인한다.

1. **5 digit addition.** random한 5 digit number pair를 sample하고 model이 합을 예측하게 한다.
2. **Repeated subsequence 예측.** uniform random token sequence를 만들고 repeat할 subsequence를 random하게 고른 뒤, model이 반복 token을 예측하도록 학습한다.
3. **Skip trigram.** 0~19 token sequence를 주되 정확히 하나만 10 이상이며, model이 ≥ 10 token을 output해야 한다. 이는 10개 skip trigram 학습으로 풀 수 있다.

5-digit addition에는 1-layer full transformer, repeated subsequence 예측에는 2-layer attention-only transformer, skip trigram task에는 1-layer attention-only transformer를 사용했다. 그 외 hyperparameter는 mainline model과 동일하다.

5 Digit Addition

먼저 거의 infinite data regime에서 training하는 case를 본다. 각 minibatch마다 random 5 digit number를 새로 sample한다. Figure 29의 결과에서 train loss와 test loss가 일치하므로 grokking은 일어나지 않는다. 가능한 pair가 10^{10} 개라 model이 같은 5 digit number pair를 두 번 볼 일이 거의 없기 때문이다. 흥미롭게도 Figure 29의 여러 작은 bump는 model이 output의 6개 token 각각을 계산하는 방법을 학습하는 데 대응한다. 그러나 Figure 30처럼 model이 700 data point만 보도록 제한하면 grokking이 일어난다.

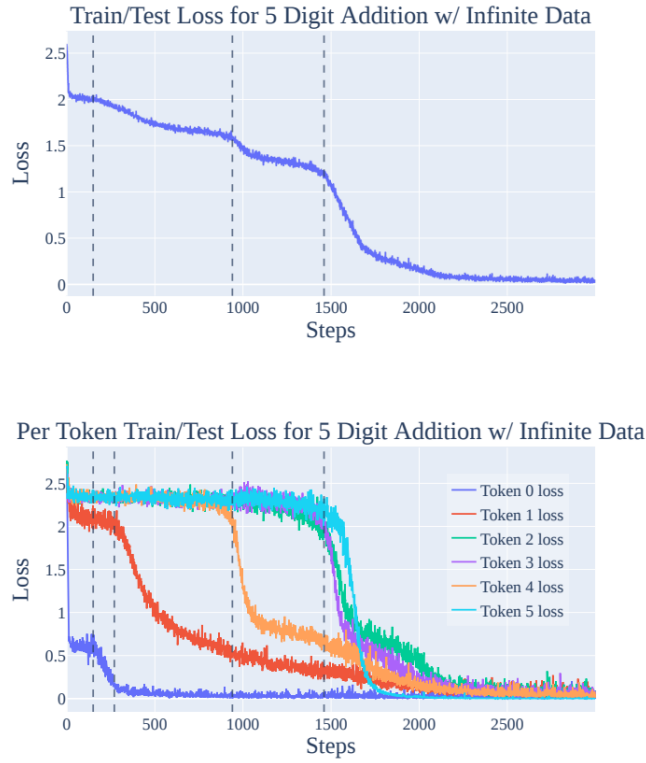


그림 29. (위) random-generated data로 학습한 5 Digit Addition의 train/test loss. model이 반복 pair를 보지 않으므로 train/test loss가 일치한다. (아래) 매 step random-generated data로 학습할 때 5 Digit Addition의 token별 train/test loss. average loss의 phase change는 individual token phase change에 대응한다. 다만 token 1의 약 step 270 phase change는 token 0 첫 phase change의 끝과 겹쳐 average loss에서 드러나지 않는다.

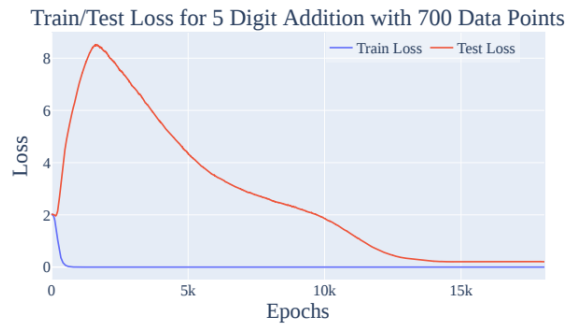


그림 30. 700 data point로 학습한 5 Digit Addition의 train/test loss. infinite random-generated data case와 달리 sharp phase change와 뚜렷한 train/test divergence를 보인다.

Repeated subsequence

5-digit addition과 같이 repeated subsequence task에서도 data amount 제한은 grokking에 필요하고 충분하다. Figure 31에서 매 step 새로운 data를 보는 model은 grokking을 보이지 않는다. 반면 Figure 32에서 model이 512 data point만 보도록 제한하면 뚜렷한 grokking이 발생한다.

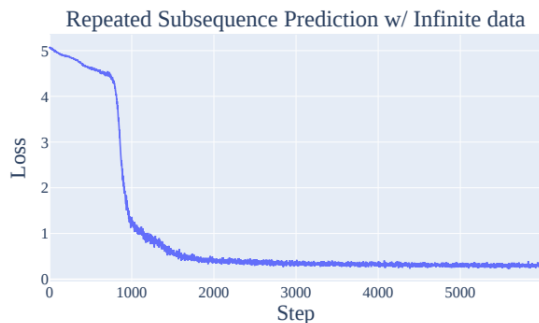


그림 31. random-generated data로 학습한 repeated subsequence의 train/test loss. repeated pair를 보지 않으므로 train/test loss가 일치한다. sharp phase change는 model이 induction head를 형성하는 데 대응한다(Olsson et al., 2022).

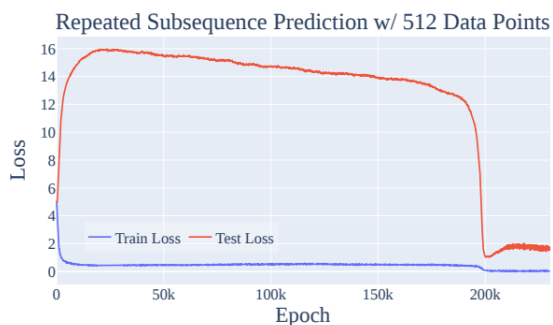


그림 32. 512 data point로 학습한 repeated subsequence task의 train/test loss. infinite random-generated data case와 달리 sharp phase change와 뚜렷한 train/test divergence를 보인다.

Skip trigram

앞 task들과 마찬가지로 skip trigram에서도 data amount 제한은 grokking에 필요하고 충분하다. 매 step 새 data를 보는 model은 Figure 33에서 grokking을 보이지 않으며, 512 data point만 보는 model은 Figure 34에서 명확한 grokking을 보인다. 종합하면 이 결과는 grokking에서 limited data의 중요성을 재확인한다.

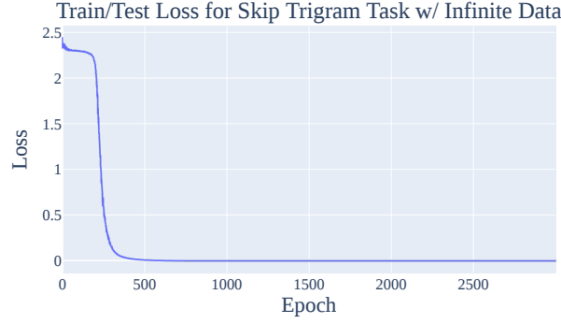


그림 33. random-generated data로 학습한 skip trigram task의 train/test loss. repeated pair를 보지 않으므로 train/test loss가 일치한다. sharp phase change는 network가 모든 skip trigram을 학습하는 데 대응한다.

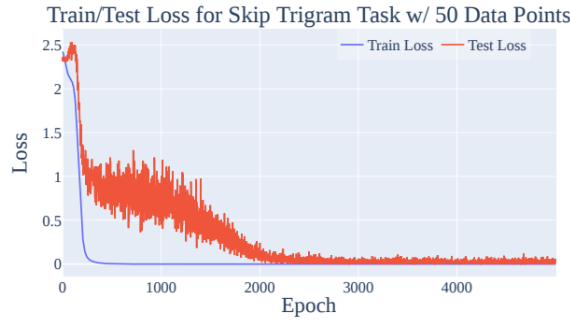


그림 34. 512 data point로 학습한 skip trigram task의 train/test loss. infinite random-generated data case와 달리 sharp phase change와 뚜렷한 train/test divergence를 보인다.

APPENDIX E. GROKking에 대한 추가 추측

E.1. Grokking의 직관적 설명

이 절에서는 model이 grokking할 때 “under the hood”에서 무엇이 일어날 수 있는지, 그리고 이 현상이 왜 발생하는지를 추측한다. evidence는 suggestive할 뿐이므로 향후 연구의 유망한 방향이다.

Grokking은 특정 hyperparameter로 algorithmic task에서 학습한 model이 처음에는 training data를 overfit할 때 일어난다. 즉 train loss는 크게 개선되지만 test loss는 나빠져 둘 사이가 diverge한다. 그러나 training 후반에는 test loss가 갑자기 개선되어 test/train loss가 converge한다. Power et al. (2022)과는 다르지만 Liu et al. (2022)와 일치하게, Figure 2와 18을 포함한 본 논문의 여러 figure처럼 initial divergence 없이 train/test loss가 함께 개선되는 경우에는 grokking이 일어나지 않는다.

핵심 문제는 model에 두 가능한 solution이 있다는 것이다. 하나는 memorization(low train loss, high test loss), 다른 하나는 generalization(low train loss, low test loss)이다. 이 case에서 Fourier multiplication algorithm이 generalization solution이다. 직관적으로 training data가 매우 적으면 model은 overfit하고 memorize한다. data가 더 많아지면 model은 generalize하지 않으면 train/test loss 모두 나빠진다. Neural network는 “더 단순한” solution을 선호하는 inductive bias를 가지므로 memorization complexity는 training set 크기에 따라 scale하지만 generalization complexity는 constant다. 둘은 어느 지점에서 반드시 교차한다. 그럼에도 grokking에서 놀라운 점은, training 중 model이 memorization에서 generalization으로 전환할 때의 abrupt shift다.

Grokking의 다른 component는 phase transition이다. 이는 특정 task에 학습된 model이 짧은 training 기간에 특정 capability를 꽤 빠르게 획득하는 현상이다. Olsson et al. (2022)의 transformer language model에서 induction head 형성, Appendix D.3의 본 연구 결과가 예다. 즉 capability를 training 내내 천천히 형성하기보다, model은 빠르게 그것을 못하는 상태에서 잘하는 상태로 간다. phase transition의 한 해석은 loss landscape에 generalizing solution 접근을 어렵게 하는 feature가 있다는 것이다. model이 따를 smooth gradient가 아니라 처음에는 progress가 어렵지만 어떤 threshold를 넘으면 빠르게 진전한다.

따라서 grokking은 phase transition, limited data, regularization과 함께 발생한다. model은 overfitting을 피하기에 충분한 training data가 있어도 phase transition을 보인다. regularization(본 연구에서는 weight decay)은 complex solution보다 simple solution을 선호한다. model에는 memorization보다 generalization을 marginally 선호할 만큼의 data가 있다. 그러나 phase transition은 memorization에는 문제가 없는 반면 generalization이 “도달하기 어렵다”는 점을 가리킨다. model이 memorize하면서 network는 더 complex해지고, 어느 시점에 weight decay가 추가 memorization을 막아 equilibrium으로 이동한다. memorize하려는 gradient는 smaller weight를 향하는 gradient와 balance한다. generalization에서는 model이 memorize와 simplify 양쪽 모두에 incentive를 갖는다. 두 행동을 동시에 수행하면서 circuit formation phase에 training performance를 어느 정도 constant하게 유지할 수 있다는 점이 눈에 띈다.

다음으로 model이 generalization에 접근하면 cleanup phase에서 memorization weight가 제거된다. complexity cost가 lower loss benefit보다 커진다. 이 training period의 phase transition으로 generalization을 향한 progress가 가속되면서 cleanup rate도 가팔라진다.

perfect solution을 학습하고 weight decay로 training된 model에는 더 extreme한 logit과 더 낮은 loss를 위한 larger weight, 그리고 weight decay로 인한 smaller weight라는 경쟁 incentive가 있다. 그러므로 어떤 solution 및 weight decay 수준에 대해서도 두 힘이 equilibrate하는 train loss 수준이 존재한다. 따라서 memorization이 generalization보다 반드시 “더 단순한” solution인 것은 아니다. 핵심은 train loss를 고정했을 때 generalization의 weight가 더 작다는 점이다. base solution이 더 단순하므로 weight decay는 generalization에서 약간 더 낮은 train loss에서 equilibrate할 것으로 예상된다. 이는 실제 관찰과 맞는다.

주: grokking은 종종 “model이 zero loss를 달성한 뒤에도 generalize를 학습했다”라고 부정확하게 요약된다. cross-entropy loss에는 zero loss가 존재하지 않는다. model은 perfect accuracy에 도달해도 accuracy가 아니라 loss를 optimize하므로 계속 개선할 incentive가 있다. perfect accuracy에서 성능을 개선하는 가장 쉬운 방법은 logit scale을 올려 temperature를 낮추고 softmax를 argmax에 더 가깝게 만드는 것이다.

E.2. 가설: Phase transition은 composition에 내재한다

성장 중인 mechanistic interpretability 분야의 유망한 연구는 model이 circuit(Cammarata et al., 2020)을 형성함을 시사한다. circuit은 image classification network의 curve detector(Cammarata et al., 2020), LLM의 induction head(Elhage et al., 2021; Olsson et al., 2022)처럼 model subnetwork로 구성된, 깔끔하고 해석 가능한 algorithm이다. 이는 놀라운 사실이다. circuit은 model이 algorithm, 즉 근본적으로 discrete한 것을 학습한다는 뜻이다. algorithm의 각 step은 다른 step이 존재할 때만 의미가 있다. 하지만 Neural network는 근본적으로 continuous하며, 낮은 loss를 향하는 gradient를 따라 학습되고 smooth gradient를 따르지 않고 새 optimum으로 jump하기 어렵다. 그렇다면 model은 어떻게 discrete algorithm을 학습할 수 있을까?

구체적 예로 LLM의 induction head를 보자. next-token prediction autoregressive language model에는 repeated subsequence를 계속 이어 가는 subnetwork가 있다. current token이 context 초기에 발생했는지 탐지하고, 그렇다면 그 이전 발생 뒤에 왔던 동일 token도 다음에 올 것이라고 예측한다. circuit은 각 previous token에 attention하여 previous token context를 current token으로 copy하는 previous token head와, current token의 이전 occurrence 다음 token에 attention하는 induction head로 구성된다. induction head는 current token을 representation하는 query vector와 previous token head output, 즉 previous token context를 representation하는 key vector를 K-Composition으로 형성하여 previous token head와 compose한다. 이어 이 query/key가 match하는 token에 attention한다.

이 circuit은 loss를 크게 개선하지만 다른 head가 존재하는 context에서만 그렇다. 어느 head도 존재하지 않으면 어느 head 형성도 독려하는 gradient가 없다. initialization에는 두 head 모두 없으므로, gradient descent가 이 circuit을 발견하지 못할 것이라고 생각할 수 있다. 순진하게는 Neural network가 linear regression과 비슷한 circuit, 즉 continuous training 중 각 weight가 marginally performance를 개선하는 circuit만 만들 것이라고 예측할 수 있다. 그러나 실제 Neural network는 여러 부분이 non-trivial하고 algorithmic한 방식으로 상호작용하는 정교한 circuit을 형성한다. 어떻게 가능한가?

가능한 설명은 몇 가지 있다.

- **Lottery tickets (Frankle & Carbin, 2018).** 초기에는 network 각 layer가 많은 partial circuit component의 superposition이고 각 layer output은 component output 평균이다. network full output은 서로 다른 많은 circuit의 평균이며 non-linear interaction에 따른 큰 interference가 있다. 그중 일부 circuit은 loss를 낮추는 데 systematic하게 유용하지만 대부분은 그렇지 않다. useless circuit의 gradient는 zero mean인 반면 useful circuit의 gradient는 noise가 많지만 non-zero mean이다. SGD는 relevant circuit을 reinforce하고 useless circuit을 suppress하므로 circuit은 점진적으로 형성된다.
- **Random walk.** network는 loss landscape를 무작위로 돌아다니다가 어느 정도 compose되는 반쯤 형성된 previous token head와 induction head를 만난다. 이 half-formed circuit은 loss를 낮추는 데 유용해지고 gradient descent가 circuit을 완성한다.
- **Evolution.** 인간의 눈 같은 정교한 machinery를 organism이 어떻게 발달시키는가에도 비슷한 수수께끼가 있다. 각 부분은 다른 부분의 context에서만 유용하다. 그럴듯한 설명은 light-detecting membrane처럼 단독으로도 어느 정도 유용한 component가 먼저 발달해 useful component로 reinforce되고, 이후 eye lens처럼 첫 component에 의존하는 부분이 생겼다는 것이다.

Evolution은 자연스러운 설명이지만, toy task를 보면 전체 이야기는 될 수 없다. repeated subsequence task에서는 임의 location의 repeated subsequence를 제외하면 uniform random token sequence가 있다. 예를 들어 7 2 8 3 1 9 3 8 3 1 9 9 2 5 END처럼 모든 token pair는 repeated subsequence의 같은 token pair를 제외하면 independent다. 특히 previous token head는 current token loss를 절대 줄일 수 없다. previous token은 항상 next token과 independent다. 그러므로 previous token head는 circuit을 완성하는 induction-like head의 context에서만 유용하다. 마찬가지로 induction head는 previous token head와의 K-composition에 의존하므로 단독으로는 유용할 수 없다. 그런데도 model은 결국 induction circuit을 형성한다.

a priori에는 random walk만으로 충분히 보이지 않는다. induction circuit은 model space의 작은 region을 차지하는 비교적 복잡한 구조이므로 random walk가 우연히 맞닥뜨리기 어렵다. modular addition case에서는 progress measure가 pre-grokking에 유의미한 hidden progress를 보이므로, model이 우연히 solution을 만난 것이 아님을 더 구체적으로 알 수 있다.

따라서 lottery ticket hypothesis가 가장 설명력이 있어 보인다. induction head는 previous token head 없이는 useless하지만, previous token에 uniform하게 attention하는 head와 compose하면 output 일부에 previous token이 포함되므로 약간 유용할 수 있다. 그래도 모든 explanation이 전체 그림에 기여할 것이라고 본다. uniform head가 random walk를 통해 previous token에 조금 더 attention하게 되는 경우라면 특히 그럴 듯하다.

phase transition으로 돌아가면, lottery ticket 스타일 explanation은 circuit이 형성되면서 phase transition이 예상될 수 있음을 시사한다. circuit formation 초기에는 각 circuit 부분이 조잡하여 component 하나를 개선해도 loss 효과가 약하고 gradient가 작다. 각 component가 발달하면 다른 component가 더 유용해져 모든 gradient가 non-linear하게 함께 커진다. circuit completion에 가까워지면 해당 circuit loss curve의 acceleration이 나타나 phase transition이 발생할 것으로 기대한다.

APPENDIX F. EMERGENT PHENOMENA 연구에 mechanistic interpretability와 progress measure를 사용하는 방법에 대한 추가 논의

mechanistic interpretability로 progress measure를 정의하는 접근은 비교적 유망하지만, 기존 mechanistic interpretability approach가 실제로 얼마나 scalable한지는 여전히 큰 불확실성이 있다. 향후 mechanistic interpretability 연구 성공 여부에 따라, mechanistic interpretability와 progress measure가 emergent phenomenon을 이해·예측하는 데 도움이 되는 경로는 크게 세 가지다.

1. mechanistic interpretability가 large model로 scale되어 behavior의 유의미한 부분 뒤 mechanism을 이해할 수 있다면, 본 연구와 같은 방식의 analysis를 수행할 수 있다. mechanistic interpretability가 이런 정도로 large model에 성공적으로 scale될지(또는 정교한 behavior 전부에 human-understandable explanation이 존재하는지) 현재는 불분명하다. 다만 human-understandable mechanism을 회복하는 경우, mechanism 부분을 그대로 progress measure로 쓸 수 있다.

2. future mechanistic interpretability가 Wang et al. (2022)처럼 larger model mechanism의 일부만 회복하고, smaller model에서는 mechanism의 포괄적 이해만 제공할 수 있다 해도, smaller model 이해를 이용해 larger model behavior 일부를 추적하는 measure 개발을 이끌 수 있다. 기존 mechanistic interpretability 연구가 이미 large model behavior fragment를 회복하고 smaller model과의 analogy로 이해하게 해 주므로, 이는 비교적 그럴듯한 scenario다. 예를 들어 Olsson et al. (2022)은 이 approach로 medium-sized language transformer에서 in-context learning emergence를 이해한다.
3. mechanistic interpretability가 large model에서 human-understandable mechanism을 전혀 회복하지 못해도, human understanding이 필요 없는 progress measure를 도출할 수 있을지 모른다. 예를 들어 automated mechanistic interpretability가 결국 human-understandable mechanism을 회복하지 못하는 opaque process가 되더라도, 그 output을 사용할 수 있을 것이다.

또 다른 방향은 task-independent progress measure다. 여러 small, interpretable model을 testbed로 이용해 task에 의존하지 않는 progress measure를 발견할 수 있다면 이를 large model에 적용할 수 있을 것이다.

다만 Section 6에서 제시한 future work가 기존 large language model의 emergent behavior를 예측하고 이해하는 데 이 접근을 성공적으로 적용하는 데 필요하다고 생각하며, 따라서 우리는 신중한 낙관론을 유지한다.

REFERENCES

참고문헌의 저자명·논문명·출판 정보는 식별성과 정확성을 위해 원문 English 표기를 유지했습니다.

Boaz Barak, Benjamin L Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Hidden progress in deep learning: Sgd learns parities near the computational limit. arXiv preprint arXiv:2207.08799, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Nick Cammarata, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, Chelsea Voss, Ben Egan, and Swee Kiat Lim. Thread: Circuits. *Distill*, 2020. doi: 10.23915/distill.00024. <https://distill.pub/2020/circuits>.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.

Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. arXiv preprint arXiv:1803.03635, 2018.

Deep Ganguli, Danny Hernandez, Liane Lovitt, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova Dassarma, Dawn Drain, Nelson Elhage, et al. Predictability and surprise in large generative models. In *2022 ACM Conference on Fairness, Accountability, and Transparency*, pp. 1747–1764, 2022.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant,

Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2.

Niall Hurley and Scott Rickard. Comparing measures of sparsity. *IEEE Transactions on Information Theory*, 55(10):4723–4741, 2009.

Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J Michaud, Max Tegmark, and Mike Williams. Towards understanding grokking: An effective theory of representation learning. *arXiv preprint arXiv:2205.10343*, 2022.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Thomas McGrath, Andrei Kapishnikov, Nenad Tomasev, Adam Pearce, Demis Hassabis, Been Kim, Ulrich Paquet, and Vladimir Kramnik. Acquisition of chess knowledge in AlphaZero. *arXiv preprint arXiv:2111.09259*, 2021.

Beren Millidge. Grokking ‘grokking’. 2022. <https://www.beren.io/2022-01-11-Grokking-Grokking/>.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544*, 2022.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Plotly Technologies Inc. Collaborative data science. 2015. <https://plot.ly>.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Alex Rogozhnikov. Einops: Clear and reliable tensor manipulations with einstein-like notation. In *International Conference on Learning Representations*, 2022. <https://openreview.net/forum?id=oapKSVM2bcj>.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

Jacob Steinhardt. More is different for AI. Feb 2022. <https://bounded-regret.ghost.io/more-is-different-for-ai/>.

Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roni Paiss, and Joshua Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *arXiv preprint arXiv:2206.04817*, 2022.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 Small. *arXiv preprint arXiv:2211.00593*, 2022.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022a.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022b.

Wes McKinney. Data Structures for Statistical Computing in Python. In Stéfan van der Walt and Jarrod Millman (eds.), *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.